

Revisiting DKLs Threshold ECDSA: Enhanced OT-based VOLE and Two-Party Signing

Gilad Asharov*
Utila

May 17, 2026

Abstract

Threshold ECDSA signing has become a standard building block for securing cryptocurrency assets, with the protocol of Doerner, Kondi, Lee, and Shelat (DKLs, IEEE S&P 2024) emerging as a leading solution due to its efficiency and widespread industry adoption.

In this work, we revisit the DKLs protocol to evaluate its concrete security and implementation trade-offs:

- **Vector Oblivious Linear Evaluation (VOLE):** We identify subtle issues in the underlying OT-based Vector Oblivious Linear Evaluation (VOLE) sub-protocol, showing that original parameter choices must be adjusted to reach intended security levels. To address this, we provide a complete analysis of three VOLE variants offering different trade-offs between bandwidth and round complexity.
- **Two-Party Signing:** We introduce an optimized two-party signing protocol that shifts the majority of computation and communication to a message- and key-independent pre-processing phase. This results in an exceptionally efficient online phase where each party exchanges only 0.2KB, a roughly 600× reduction in communication compared to the full protocol, without being susceptible to known “pre-signature” attacks.

Our findings consolidate the security of the protocol while providing significant efficiency improvements for practical deployment and standardization.

*The author is also at Bar-Ilan University. Research was conducted for Utila. Gilad@utila.io, Gilad.Asharov@biu.ac.il

Contents

1	Introduction	2
2	Technical Overview	3
2.1	Multiplication (Oblivious Linear Evaluation)	3
2.2	Two-Party Signing	6
2.3	An Efficient Attack in the Endemic OT Hybrid Model	8
2.4	Summary of Our Contributions	9
2.5	Related Work	9
3	Preliminaries	10
3.1	ECDSA Signatures	10
3.2	Building Blocks	11
4	Random Vector Oblivious Linear Evaluation	12
4.1	Functionality	12
4.2	The Protocol	12
4.2.1	Variant II: Full Description	13
4.3	Variant I: Simpler Protocol – More Rounds	15
4.4	Variant III: The Protocol of [DKLs24]	15
4.5	Attack on the Protocol in the Endemic OT-Hybrid Model	16
4.6	Analysis	18
5	Applying Leftover Hash Lemma: Various Games and their Required Parameters	18
5.1	Variant I: Plain Leftover Hash Lemma	18
5.2	Variant II: Leftover Hash Lemma with Adversarially-Limitedly-Chosen Hash	19
5.3	Variant III: Leftover Hash Lemma with Adversarial Source-Fixing	22
5.3.1	Instantiation in Our Setting	23
6	The ECDSA Two-Party Signing Protocol	24
6.1	The Functionality	24
6.2	The Protocol	24
7	Evaluation and Efficiency	27
	References	28
A	Experiment on Small Parameters	32
B	Full Proofs for the VOLE Protocols	32
B.1	Security Analysis for Variant I	32
B.1.1	Correctness	33
B.1.2	Malicious Sender	33
B.1.3	Malicious Receiver	41
B.2	Variant II: Compressing Rounds – Security Analysis for Protocol 4.2	44
B.3	Variant III: The Protocol of [DKLs24]	46
C	A Full Proof for the Two-Party Signing Protocol	46

1 Introduction

ECDSA is the dominant signature scheme used to control cryptocurrency assets, securing transactions worth billions of dollars daily. To mitigate single points of failure, modern systems increasingly rely on t -out-of- n threshold ECDSA, where any subset of t parties can jointly produce a valid signature, while no coalition of at most $t - 1$ parties can do so, even when interacting with honest participants. Threshold ECDSA has therefore become a standard building block in deployed cryptographic wallet infrastructures.

At a high level, the parties hold shares $\mathbf{sk}_i$ of a secret key $\mathbf{sk}$ corresponding to a public key $\mathbf{pk}$, and collaboratively compute a signature $\sigma = \text{Sign}_{\mathbf{sk}}(m)$ on a message m . Recent years have seen significant progress in designing efficient threshold ECDSA protocols (see, e.g., [Lin17, HLN18, DKLs18, CGG⁺20, KU25] and references therein). Among these, the protocol of Doerner, Kondi, Lee, and shelat (DKLs) [DKLs24], building upon previous protocols by the same authors [DKLs18, DKLs19], stands out for its conceptual clarity, elegance, and practical efficiency. It has been adopted in several real-world systems, including companies such as Coinbase, Silence Laboratories, Alore/0xCarbon, Taurus Group, Utila, Vultisig, Copper, Sodot, Web3auth, and more (see [DKL25]).

Our perspective and contributions. Motivated by the increasing deployment of the DKLs protocol, we revisit its design and analysis with the goal of understanding its concrete security guarantees and implementation trade-offs in the plain model. At a high level:

- **OT-Based Multiplication (VOLE):** We identify several subtle issues arising from the interaction between the protocol’s building blocks, most notably in the vector oblivious linear evaluation (VOLE) sub-protocol, which is instantiated from oblivious transfer (OT). Our analysis shows that the original parameter choices must be adjusted in order to fully meet the protocol’s intended security levels. In particular, implementations that follow the protocol exactly as described in [DKLs24] may fail to achieve their targeted security guarantees. Achieving the desired level of security requires either increased communication bandwidth or additional interaction rounds. We provide a complete analysis of three protocol variants, each offering a different trade-off between bandwidth and round complexity.
- **Fast Two-Party Signing:** The two-party setting already captures the core threshold-signing model: the signing key is shared between two parties, so neither party can generate signatures on its own. This distributes trust and removes any single point of key compromise. This setting is also the focus of several wallet providers. We therefore optimize the two-party signing protocol of [DKLs24]. Our starting observation is that several steps in the multiparty signing protocol of [DKLs24] become unnecessary in the two-party setting. By eliminating these redundancies, we reduce both communication and round complexity, while still accounting for the additional interaction required by the strengthened underlying VOLE protocol. We further show that a large fraction of the protocol’s interaction and computation can be shifted to a preprocessing phase. This preprocessing produces an extremely small state of less than 0.3KB. The subsequent online phase is highly efficient: each party sends and receives only about 0.2KB, and performs less than 1% of the computation required in preprocessing. This stands in sharp contrast to a direct execution of the full protocol, which requires more than $\approx 600\times$ as much communication, with each party sending approximately 200KB. Importantly, the preprocessing phase is independent of both the message m and the secret key $\mathbf{sk}$, and the

nonce is not revealed prior to choosing the message to be signed, thereby avoiding the known attacks on “pre-signatures” identified by Groth and Shoup [GS22].

Our work both consolidates the security of the protocol and suggests direct efficiency improvements. Because this protocol is already deployed in systems securing substantial financial value, even subtle mismatches between theoretical analysis and practical implementation can have significant consequences. Furthermore, the protocol is a strong candidate for standardization by NIST. We believe our work contributes to this effort by helping to formalize a highly secure, correctly parameterized instantiation. More broadly, our analysis highlights several intriguing theoretical questions with direct implications for practical schemes, which may inspire further research.

2 Technical Overview

The construction of [DKLs24] suggests a three-round protocol for generating a signature. However, realizing this round complexity in the plain model is non-trivial. At a high level, the construction provides: (a) a threshold signing protocol given a multiplication protocol; and (b) an instantiation of a multiplication protocol given an oblivious transfer on random inputs. We start with an overview of each component separately.

2.1 Multiplication (Oblivious Linear Evaluation)

In Vector Oblivious Linear Evaluation (VOLE), the **Sender** holds $(a_1, \dots, a_\ell) \in \mathbb{Z}_q^\ell$, while the **Receiver** holds $\beta \in \mathbb{Z}_q$, and the goal is to obtain additive shares of $(a_1 \cdot \beta, \dots, a_\ell \cdot \beta)$ over $\mathbb{Z}_q$.

VOLE can be viewed as an arithmetic analogue of oblivious transfer (OT). In OT, the **Sender** holds two strings (x_0, x_1) and the **Receiver** holds a choice bit σ , learning only x_σ . VOLE replaces this binary selection with a linear correlation over a field. The construction of [DKLs24] realizes VOLE using OT as a subroutine with random inputs, where the **Receiver**’s value β is (implicitly) sampled uniformly by the protocol. Thus, implementing VOLE reduces to instantiating OT on random inputs.

Several such random OT variants have been studied, including Sender Random OT, Random Receiver OT [ALSZ17], and Endemic OT [MR19]. Among these, Endemic OT offers the most flexible abstraction and admits a single-round realization. Moreover, the number of OT instances required in [DKLs24] is relatively small (e.g., about 376 for a statistical security parameter of 60). Hence, despite the reliance on public-key operations, directly using Endemic OT may be preferable to OT extension techniques, which introduce an additional round of interaction. In particular, one-round Endemic OT yields a two-round VOLE protocol, consequently enabling a three-round signing protocol.

However, a subtle issue in [DKLs24] complicates this picture. We observe that the suggested number of OTs does not suffice for the claimed security level. Our analysis indicates that the number of required OTs must be increased significantly (from roughly 376 to approximately 1400). This leads to roughly a $4\times$ increase in communication, with each signature requiring at least 400KB of bandwidth. Moreover, directly instantiating this many OTs using public-key cryptography entails substantial computational overhead, complicating direct implementations via Endemic OT.

On the number of oblivious transfers needed. To detail why the required number of OTs must be increased, we review how the random input $\beta \in \mathbb{Z}_q$ of the VOLE is sampled in [DKLs24]:

The two parties sample a “gadget vector” $\mathbf{g} \in \mathbb{Z}_q^m$ at the outset of the computation, derived from shared randomness, where m is the number of OTs. The **Receiver** samples $\mathbf{w} \in \{0, 1\}^m$ as their random input to the oblivious transfer. The input $\beta \in \mathbb{Z}_q$ is then derived as $\sum_{i=1}^m g_i \cdot w_i$. The vector $\mathbf{g}$ can be viewed as a hash function mapping $\{0, 1\}^m \rightarrow \mathbb{Z}_q$, denoted as $\mathbf{g}(\mathbf{w})$.

The leftover hash lemma [ILL89] guarantees that if (1) a random hash function $\mathbf{g} : \{0, 1\}^m \rightarrow \mathbb{Z}_q$ is sampled from a universal family, and (2) $\mathbf{w} \leftarrow \{0, 1\}^m$ is drawn independently from a source W with sufficiently high min-entropy, then $\beta = \mathbf{g}(\mathbf{w})$ is close to uniform over $\mathbb{Z}_q$. The exact parameters of the lemma determine the required min-entropy of W , which in turn dictates the number of oblivious transfers m (directly impacting communication complexity and efficiency). Crucially, $\mathbf{g}$ and $\mathbf{w}$ must be chosen independently.

The analysis of [DKLs24] treats $\mathbf{g}$ and $\mathbf{w}$ as independent and applies the leftover hash lemma¹ to deduce that β is 2^{-s} -close to uniform when $m \geq \log q + 2s$ for a statistical security parameter s . However, in the actual protocol, the setting differs subtly: Because $\mathbf{g}$ is chosen at the outset of the computation, an adversarial **Sender** can leverage this to carefully restrict the valid choices of $\mathbf{w}$ (assuming no abort occurs in the consistency check phase). In other words, the adversary can adaptively choose which min-entropy source W over $\{0, 1\}^m$ to use *after* observing $\mathbf{g}$.

While the leftover hash lemma guarantees that a random $\mathbf{g}$ behaves well for any *fixed* high-min-entropy source, it does not rule out the possibility that for a *fixed* $\mathbf{g}$, there exists an adversarial chosen high-min-entropy source $\tilde{W}$ for which $\mathbf{g}(\tilde{W})$ is not sufficiently close to uniform.

We demonstrate that the original proof does not fully encompass this adaptive strategy under the proposed parameters. To validate this, we conduct an experiment (detailed in Appendix A) using small parameters to test whether the condition $m \geq \log q + 2 \cdot s$ is sufficient. By quantifying over all possible $\mathbf{g}$ and sources W , our results indicate that there indeed exist specific $\mathbf{g}$ ’s that admit “bad” sources W , causing the statistical distance to exceed the theoretical bound. This confirms that the initial parameter choice requires adjustment.

Three variants. Motivated by this observation, we propose three robust alternatives for the protocol instantiation.

Variant I: Standard Leftover Hash Lemma. We begin by slightly modifying the protocol to fit cleanly into the standard setting of the leftover hash lemma, ensuring the hash function $\mathbf{g}$ and the source W are chosen independently. We provide this variant primarily for pedagogical purposes to aid understanding. Because the randomness of $\beta = \mathbf{g}(\mathbf{w})$ is meant to protect against an adversarial **Sender**, we can simply have the **Receiver** pick $\mathbf{g}$ uniformly at random at the *end* of the protocol and send it to the **Sender**. With this adjustment, the leftover hash lemma applies directly, allowing for small parameters: $m \geq \log q + 3 \cdot s$ (roughly 436 OTs for $s = 60$).

However, this variant introduces two practical trade-offs: (1) It adds an extra communication round (to send $\mathbf{g}$), and (2) the **Receiver** obtains the value $b = \mathbf{g}(\mathbf{w})$ only after receiving the second message, rather than at the end of the first round. Consequently, β is only available to the outer protocol at the very end of the VOLE execution, see a discussion on that below.

Variant II: Leftover Hash Lemma with Mildly Adversarial Hash Choice. To avoid adding an extra message, we explore a Fiat-Shamir-like transformation. In this approach, we derive $\mathbf{g}$ by applying a random oracle to the final message from the **Sender**. This means $\mathbf{g}$ is based (indirectly) on the source W already fixed by the **Sender**.

¹In [DKLs24], the lemma is attributed to Proposition 1.1 of Impagliazzo and Naor [IN96].

This approach provides slightly more power to an adversarial **Sender** compared to Variant I. The **Sender** could theoretically test multiple candidate sources W , observe the resulting hashes $\mathbf{g}$, and select the most advantageous combination. We show that this extra power can be safely mitigated by increasing the number of OTs by an additive factor of 2κ , where κ is the computational security parameter (typically 128). This requires $m \geq \log q + 3 \cdot s + 2 \cdot \kappa$. Concretely, for $s = 60$ and $\kappa = 128$, this totals 692 OTs (rounded in implementations to 696 to be a multiple of 8).

While this approach preserves the original round complexity of the VOLE, the **Receiver** still learns β late in the protocol. Because the signing protocol originally requires β at the end of the first VOLE round, directly plugging in this variant would add a round to the signing protocol. However, as we will demonstrate in Section 6, we can modify the two-round signing protocol so that β is not needed early on, making it fully compatible with this VOLE variant.

Variant III: Leftover Hash Lemma with Adversarial Source-Fixing. The third variant aligns exactly with the security game of the original [DKLs24] protocol, where $\mathbf{g}$ is sampled first, and the adversary subsequently chooses a $\mathbf{g}$ -dependent high-min-entropy source W . We prove that, provided the number of OTs is sufficiently large, the vast majority of hash functions $\mathbf{g}$ are “good”—meaning that for *every* high-min-entropy choice of W , the distribution $\mathbf{g}(W)$ remains close to uniform. Achieving this guarantee, however, requires satisfying $m \geq \log q + O(s \cdot \log m)$. This results in the previously mentioned ≈ 1400 OTs.

We emphasize that our analysis here is conservative. We apply the leftover hash lemma directly, without overly optimizing constants or relying on complex bounds. **Our analysis is intentionally simple:** it relies on the leftover hash lemma and a basic union bound. We believe that in practice, transparent and easily verifiable analysis is often preferable for high-assurance systems, even if it yields slightly larger parameters.

Analysis. The adversarial source-fixing issue also affects previous versions of the protocol [DKLs18, DKLs19]. We provide full, rigorous security proofs for all the three variants we mentioned above. For clarity, our analysis first applies to a protocol with extra communication rounds (Variant I) to distill the exact properties needed, and we then apply a round-compressing transformation. Our proof is naturally based on that of [DKLs24], but has a somewhat different structure. For instance, while [DKLs24] has to rely on the leftover hash lemma in three different places, we apply it only once.

Implications. Increasing the number of OTs has important implications for practical deployments. With a larger number of OTs, it is natural to use OT extension protocols, which reduce the reliance on expensive public-key operations to a small number of base OTs (typically on the order of the computational security parameter, e.g., 128 or 256). From these, the necessary ≈ 1400 additional OTs can be generated using inexpensive symmetric-key operations.

In particular, random OT extension protocols [ALSZ17, KOS15, Roy22] can be realized in a single round, assuming an initial setup phase that establishes the base OTs. As observed in [DKLs24], these base OTs are reusable and can therefore be amortized as part of a preprocessing phase, such as distributed key generation. Consequently, the overall round complexity can, in principle, remain at three rounds per signature when such a setup is available.

Unfortunately (and as noted in [vdP23]), even when executed honestly, OT extension protocols may leak information about the secret inputs of the base OTs, which are intended to remain hidden. Moreover, in the event of an abort, the corresponding base OTs can no longer be safely reused. This

issue becomes particularly problematic when the setup is treated as immutable key material and shared across multiple concurrent signing sessions (say, where a key is split between independent pairs of servers to increase throughput). Halting all concurrent executions can be very difficult, and it may be difficult to propagate information about an abort in one instance quickly enough to prevent unsafe reuse in parallel instances.

To address this engineering challenge, the safest solution is to assume no shared state between instances (rather than the key material) and re-execute the base OTs independently for each signature. While this approach avoids the risks associated with reuse, it comes at the cost of additional communication and increased round complexity. The increased communication and computation due to base OTs, and an additional round due to OT extension, motivates us to revisit the signing protocol and look for further optimizations, while focusing on two-party signing as it already captures the core threshold-signing model.

2.2 Two-Party Signing

Before proceeding to the threshold signing protocol, we briefly recall the standard ECDSA signature scheme and the approach of [DKLs24]. An ECDSA signature requires the following steps:

1. Choose a uniform $k \leftarrow \mathbb{Z}_q$, and let $R = k \cdot G$.
2. Let r_x be the x -coordinate of R .
3. Compute $s = k^{-1} \cdot (\mathbf{H}(m) + \mathbf{sk} \cdot r_x)$, where $\mathbf{H}$ is a hash function.
4. Output $\sigma = (r_x, s)$.

Overview of the DKLs24 signing protocol. We simplify the overview for the two-party setting. The protocol is symmetric; we provide the perspective of P_i (ignoring additional OT rounds for brevity):

1. **Phase I:** Each party P_i chooses a random $k_i \leftarrow \mathbb{Z}_q$, computes $R_i = k_i \cdot G$, and commits to R_i . The parties run two parallel VOLE sessions: one where P_i is the **Receiver** and one where P_i is the **Sender**. At the end of this round, P_i learns the random input $\beta_i \in \mathbb{Z}_q$ from the instance where it acted as the **Receiver**.
2. **Phase II:** The parallel VOLE instances conclude:
 - (a) In the first execution, P_0 (as **Sender**) inputs its share of the nonce k_0 and its share $\mathbf{sk}_0$. The **Receiver** obtains the random masking β_1 , yielding random additive shares of $k_0 \cdot \beta_1$ and $\mathbf{sk}_0 \cdot \beta_1$.
 - (b) In the second execution, P_1 (as **Sender**) inputs its shares, yielding additive shares of $k_1 \cdot \beta_0$ and $\mathbf{sk}_1 \cdot \beta_0$.

The parties also exchange decommitment information for R_0, R_1 , alongside verification data (which we omit in this overview) proving that their VOLE inputs $(k_i, \mathbf{sk}_i)$ are consistent with the public values $(\mathbf{pk}_i, R_i)$. We denote the additive shares held by P_i as $(\llbracket k_0 \cdot \beta_1 \rrbracket_i, \llbracket \mathbf{sk}_0 \cdot \beta_1 \rrbracket_i)$, and $(\llbracket k_1 \cdot \beta_0 \rrbracket_i, \llbracket \mathbf{sk}_1 \cdot \beta_0 \rrbracket_i)$.

3. **Phase III:** The parties compute the joint nonce $R = R_0 + R_1$, corresponding to $k = k_0 + k_1$ (where $R = k \cdot G$). They verify the VOLE shares against $R, \mathbf{pk}_0, \mathbf{pk}_1$. Then:
 - (a) P_0 knows k_0, β_0 and holds shares of $k_1 \cdot \beta_0$ and $k_0 \cdot \beta_1$. Respectively, P_1 knows k_1, β_1 and holds shares to the same multiplications. Using these, P_0 and P_1 compute u_0 and

u_1 respectively, where:

$$\begin{aligned} u_0 &= k_0 \cdot \beta_0 + \llbracket k_1 \cdot \beta_0 \rrbracket_0 + \llbracket k_0 \cdot \beta_1 \rrbracket_0 , \\ u_1 &= k_1 \cdot \beta_1 + \llbracket k_1 \cdot \beta_0 \rrbracket_1 + \llbracket k_0 \cdot \beta_1 \rrbracket_1 . \end{aligned}$$

Letting $\beta = \beta_0 + \beta_1$, it holds that $u := u_0 + u_1 = (k_0 + k_1) \cdot (\beta_0 + \beta_1) = k \cdot \beta$.

- (b) In a similar manner, the parties compute v_0, v_1 , which are additive shares of $v := \mathbf{sk} \cdot \beta$.
- (c) Each party derives r_x from R and computes:

$$\begin{aligned} w_0 &= H(m) \cdot \beta_0 + r_x \cdot v_0 , \\ w_1 &= H(m) \cdot \beta_1 + r_x \cdot v_1 . \end{aligned}$$

It holds that $w := w_0 + w_1 = H(m) \cdot \beta + r_x \cdot \mathbf{sk} \cdot \beta$. The parties exchange (u_i, w_i) , compute (u, w) to obtain $s = w/u$, and output signature $\sigma = (r_x, s)$.

The multiparty case. The original [DKLs24] protocol is designed for n parties. Each pair (P_i, P_j) runs two instances of VOLE protocol, one in which P_i is the **Sender** and one in which P_i is the **Receiver**. Therefore, two random maskings $\beta_{i,j}$ and $\beta_{j,i}$ are generated. Each party P_i has to correlate all the instances for which it acts as a **Receiver** to a global masking β_i . Thus, P_i must send shifts $(\beta_i - \beta_{i,j})_{j \neq i}$ during Phase II. Therefore, it is crucial that $\beta_{i,j}$ is learned in Phase I. This requirement creates friction if one wishes to use our **Variant II**, unless adding a round.

We emphasize that in the two-party case, this inter-instance correlation is unnecessary, meaning β_i can safely be learned later in the protocol. Furthermore, we show that the multiparty requirement to invoke an F_{zero} functionality (to provide shares of 0), which are necessary to hide the shares of the $\mathbf{sk}$, is unnecessary in the two-party setting.

Our modifications. As discussed, Phase I requires multiple rounds to establish base OTs, and the VOLE itself demands high bandwidth. To improve online efficiency, we push the bulk of the computation into an offline preprocessing phase that is completely independent of the message m and the secret key $\mathbf{sk}$.

The VOLE protocol of [DKLs24] is not fully random – the inputs of the **Sender** are k_i and $\mathbf{sk}_i$, while the “input” of the **Receiver** is random and is determined by the protocol. In contrast, we modify the protocol to operate on completely random inputs during preprocessing. Later, during the online signing phase, the parties send “corrections” to align the random VOLE inputs with their actual values.

Our optimized approach operates as follows:

1. **Preprocessing:** The parties choose nonces k_0, k_1 , commit to R_0, R_1 , and execute a completely random VOLE (including all necessary OTs). Note that the number of OTs is at least a few hundred. At the end of the VOLE protocol, the state of each party is extremely small: a commitment to R_{1-i} , their own k_i and its opening, four additive shares from the multiplications (i.e., four field elements), and their masking β_i .
2. **Signing phase:** Executed only when the parties agree to sign on a shared message m . The key material is also only needed at this point. The signing phase consists of two rounds:
 - (a) **Round I:** Now knowing $(k_i, \mathbf{sk}_i)$, the parties adjust the random VOLE by sending scalar corrections. Simultaneously, they complete Phase II of the original protocol: opening

commitments for R_0, R_1 and proving the verification information that they indeed use k_i and sk_i .

- (b) **Round II:** Identical to the final round of [DKLs24]. The parties verify the multiplications, exchange (u_i, w_i) , and compute and verify the signature $\sigma = (r_x, s)$.

Comparison to [DKLs24, CGG⁺20]. Our preprocessing phase differs from what is considered in the literature as “pre-signatures” [GG20, CMP20, DJN⁺20, CGG⁺20] in two dimensions:

- **The nonce is determined, but not revealed:** Previous preprocessing [GG20, CMP20, DJN⁺20, CGG⁺20] required a single round (as opposed to two rounds by our approach). However, they *revealed* the public nonce R to the parties *before* the message m was determined. This allows the adversary to choose the message m as a function of R , as pointed out by Groth and Shoup [GS22], which degrades security. Because R remains hidden in our preprocessing phase, our scheme maintains standard security guarantees (see [GS22]).
- **Independence of the secret key sk .** Our preprocessing is also independent of the secret key sk . This property is particularly useful for wallets that derive multiple addresses from the same source key material, for example, via hierarchical derivation methods such as BIP-32 [Wui12]. Since preprocessing does not depend on sk , the two devices can prepare signing information before the specific account or derived key is even selected. Other pre-signatures depend on the shares of sk .

We emphasize that our preprocessing is **non-reusable** and must be consumed per signature. Furthermore, the entire communication and computation of the online phase is small. The state each party has to store is roughly 0.3KB, and each party has to send roughly 0.2KB in the online phase. By contrast, the offline phase, which requires sending ≈ 200 KB (the exact communication depends on which variant is used).

We note that the two-party, two-message approach of [DKLs24] must still essentially execute the expensive VOLE and OT phases. Moreover, this two-message protocol assumes that the base OTs can be reused, which, as discussed above, requires additional care. If the base OTs are instead recomputed for each execution, then the protocol can no longer be implemented in two rounds.

Analysis. We prove the security of our signing protocol by assuming the parties’ inputs are valid and consistent, following the methodology established for Schnorr signatures [Lin24]. This formulation ensures the simulator knows the adversary’s share, as only one valid share exists given the public keys, and it was extracted during the key-generation. While this requires specific, compatible key-generation procedures [Lin24, DKLs18] (and is not directly applied to the relaxed key-generation procedure of [DKLs24]), compatible key-generation procedures are highly efficient and widely deployed in practice.

2.3 An Efficient Attack in the Endemic OT Hybrid Model

We conclude our technical overview by highlighting an important consideration for implementing the [DKLs24] VOLE protocol. The protocol was proven secure by [DKLs24] in the Endemic OT-hybrid model [MR19]. Endemic OT picks uniform inputs for an honest **Sender** but allows an adversarial **Sender** to choose its own inputs. The protocol relies on a challenge χ for consistency checks, derived via a Fiat-Shamir-like transformation applied to a portion of the **Sender**’s message.

We show that if the protocol is implemented exactly as abstracted in the hybrid model, an adversarial **Sender** can completely bypass the consistency check, breaking correctness in polynomial time. The adversary achieves this by deriving the challenge first and subsequently tailoring its chosen OT inputs to perfectly cancel out the challenge (see Section 4.5 for technical details).

At first glance, this resembles a standard Fiat-Shamir vulnerability where insufficient data was fed into the random oracle (e.g., [BPW12, HLPT20]). The standard fix is to hash the entire transcript. However, because the hybrid model abstracts away the OT interaction, there is no formal “OT transcript” to hash that definitively binds the **Sender**’s inputs.

To resolve this in practice, implementers must hash the entire concrete transcript of the instantiated OT. Because some implementations might overlook this nuance when translating from the hybrid model to code (and we observed a few implementations that did overlook this nuance), we aim to raise awareness of this specific attack vector. Furthermore, we prove that if one utilizes Sender Random OT [ALSZ17], which strictly enforces random inputs even for a malicious **Sender**, the protocol remains completely secure without requiring transcript hashing.

2.4 Summary of Our Contributions

We summarize our contributions as follows:

1. We identify theoretical subtleties in the security analysis of [DKLs24], particularly regarding required OT counts, leftover hash lemma parameterization, and challenge derivation in the hybrid model.
2. We propose three robust variants of the protocol. Each variant requires a different number of OTs, offering flexible trade-offs regarding round complexity and when the **Receiver**’s random input is determined.
3. We streamline the two-party signing protocol, introducing a non-reusable offline phase that maintains a compact state and enables an exceptionally fast online signing phase.
4. We provide rigorous, complete security proofs for all proposed protocols. Although most of the proofs build on those of [DKLs24], they differ in several important respects. For example, whereas [DKLs24] presents the protocol directly in its reduced-round form, we first analyze a variant with more rounds and then apply a round-compression argument to reduce the round complexity. This approach isolates the precise properties needed at each step and improves readability. In addition, the proof of the VOLE construction in [DKLs24] invokes the leftover-hash lemma three times, whereas our strategy requires only a single application.

2.5 Related Work

Threshold ECDSA signing. Threshold signing protocols have received considerable attention in recent years, driven in large part by the widespread deployment of ECDSA in cryptocurrencies and other systems where eliminating single points of key exposure is important. A long line of work has developed increasingly efficient and robust threshold-ECDSA protocols, including, e.g., [Lin17, HLN18, DKLs18, GG18, GG20, CGG⁺20, KU25], and references within.

Vector Oblivious Linear Evaluation. The VOLE protocol studied in this work is realized from OT, following the broader line of OT-based preprocessing techniques. Other works study VOLE more directly, including compressed and pseudorandom-correlation-generator approaches

to VOLE (e.g., [BCGI18]). Beyond OT-based constructions, VOLE/OLE correlations can also be obtained from public-key or algebraic assumptions, including Paillier/DCR-based pseudorandom correlation functions [OSY21], Ring-LWE-based OLE constructions [BEPU⁺20, BDM22], and related LPN/Ring-LPN-based silent-correlation-generation techniques [BCG⁺19].

3 Preliminaries

Notations. We let κ denote the security parameter. A function $\mu(\cdot)$ is *negligible* if for every positive polynomial $p(\cdot)$ and all sufficiently large κ it holds that $\mu(\kappa) < 1/p(\kappa)$. A *distribution ensemble* $X = \{X(a, \kappa)\}_{a \in \mathcal{D}, \kappa \in \mathbb{N}}$ is an infinite sequence of random variables indexed by $a \in \mathcal{D}$ and $\kappa \in \mathbb{N}$. In the context of secure computation, κ is the security parameter and $\mathcal{D}$ denotes the domain of the parties' input. Two distribution ensembles $X = \{X(a, \kappa)\}_{a \in \mathcal{D}, \kappa \in \mathbb{N}}$ and $Y = \{Y(a, \kappa)\}_{a \in \mathcal{D}, \kappa \in \mathbb{N}}$ are *computationally-indistinguishable*, denoted as $X \stackrel{c}{\approx} Y$ if for every non-uniform polynomial-time algorithm D there exists a negligible function $\mu(\cdot)$ such that for every κ and $a \in \mathcal{D}$:

$$|\Pr[D(X(a, \kappa)) = 1] - \Pr[D(Y(a, \kappa)) = 1]| \leq \mu(\kappa).$$

We say that X and Y are statistically close, denoted as $X \stackrel{s}{\approx} Y$ if the above holds for every (unbounded) algorithm D .

Throughout the document, we also use the following notations:

- $\mathbb{G}$: This is a description of a group/Elliptic curve. We assume that from $\mathbb{G}$ one knows the generator G , the order of the group, and so on. For ECDSA, our targeted group is Secp256k1.
- **Scalar**: This is the scalar group associated with the curve $\mathbb{G}$. Usually, $\mathbb{Z}_q$. We assume that from $\mathbb{G}$ one can derive **Scalar** and do not pass it as another parameter to the constructions. We use **Scalar** and $\mathbb{Z}_q$ interchangeably.

Importantly, the size of **Scalar** is usually 256-bit, while it reflects a security parameter of 128-bit.

Secure two-party computation. Our definition follows the standard definition of [Can00, Gol04] (for standalone security) and [Can01] (for UC security under concurrent composition).

3.1 ECDSA Signatures

For completeness, we provide the definition of ECDSA (see, e.g., [DKLs24]).

Construction 3.1: ECDSA signature

It assumes some group $\mathbb{G}$ for which the discrete logarithm problem is believed to be hard. Let G be the generator of $\mathbb{G}$, and **Scalar** be its associated scalar set ($\mathbb{Z}_q$). Let H be a hash function.

- **Gen**(1^κ): Choose a uniform $x \leftarrow \mathbf{Scalar}$ and set $X = x \cdot G$. The public key $\mathbf{pk} = X$ and the secret key $\mathbf{sk} = x$.
- **Sign** _{$\mathbf{sk}$} (m): On input the private key $\mathbf{sk}$ and a message m , choose a random $k \leftarrow \mathbf{Scalar}$ and let $R = k \cdot G$. Let r^x be the x -coordinate of R . Compute $s := [k^{-1} \cdot (H(m) + \mathbf{sk} \cdot r^x)]$. (If $r^x = 0$ or $s = 0$ then restart with a fresh choice of k). Output the signature $\sigma = (r^x, s)$.

- $\text{Vrfy}_{\text{pk}}(m, \sigma)$: Compute $R = (H(m) \cdot s^{-1}) \cdot G + (r^x \cdot s^{-1})\text{pk}$. Verify that the x -coordinate of R is r^x , and if so, output 1.

3.2 Building Blocks

The random oracle model. For simplicity, we assume that all the protocols are in the random oracle model. The functionality for a random oracle is defined as follows:

Functionality 3.2: The Random Oracle Functionality $\mathcal{F}_{\text{ROM}}^{\mathcal{D}}$

The functionality is parametrized by a domain $\mathcal{D}$.

- Upon receiving (ROM, x) , the functionality checks if a pair (x, y) is stored for some $y \in \mathcal{D}$. If yes, it returns y . Else, it chooses a random $y \in \mathcal{D}$, stores (x, y) , and returns y .

We sometimes use $\mathcal{F}_{\text{ROM}}^{\text{size}}$ as $\mathcal{F}_{\text{ROM}}^{\{0,1\}^{\text{size}}}$.

Commitments. Commitments can easily be instantiated using a random oracle, and implement the following functionality (see, e.g., [Lin24]):

Functionality 3.3: The commitment functionality F_{com}

The functionality F_{com} works with parties P_0, P_1 and the adversary $\mathcal{S}$, as follows:

- Upon receiving $(\text{commit}, \text{sid}, i, x)$ from party P_i ; store (sid, i, x) and send $(\text{receipt}, \text{sid}, i)$ to $\mathcal{S}$ and the honest party.
 - Upon receiving $(\text{decommit}, \text{sid}, i)$ from the party P_i , if (sid, i, x) is recorded, then send $(\text{decommit}, \text{sid}, i, x)$ to $\mathcal{S}$ and to the honest party.
-

Sender Random OT. We prove that the protocol is secure in the Sender Random OT [ALSZ17]. In this formalization, the honest **Sender** also gets random inputs from the functionality. However, malicious **Sender** cannot freely choose its inputs, and those are determined by the functionality as well.

Functionality 3.4: $\mathcal{F}_{\text{m} \times \text{RSOT}_{\mathcal{D}}}$ – Sender Random OT

The functionality interacts with the **Sender** and the **Receiver**, and with an ideal adversary $\mathcal{S}$ who can instruct to abort at any time. The domain is $\mathcal{D}$.

1. If the receiver is corrupt, then receive $(\text{receiverInput}, \text{sid}, \mathbf{w}, (\mathbf{v}_j^{w_j})_{j \in [m]})$ from the **Receiver** with $\mathbf{w} = (w_1, \dots, w_m) \in \{0, 1\}^m$ and $\mathbf{v}_j^{w_j} \in \mathcal{D}$ for every $j \in [m]$. Sample $\mathbf{v}_j^{1-w_j} \in \mathcal{D}$ for every $j \in [m]$.
2. If the sender is corrupt, or if both parties are honest, then receive $(\text{receiverInput}, \text{sid}, \mathbf{w})$ from the receiver. Sample $\mathbf{v}_j^0, \mathbf{v}_j^1 \in \mathcal{D}$ for every $j \in [m]$.

Send $(\text{senderOutput}, \text{sid}, (\mathbf{v}_j^0, \mathbf{v}_j^1)_{j \in [m]})$ to the **Sender** and $(\text{receiverOutput}, \text{sid}, \mathbf{w}, (\mathbf{v}_j^{w_j})_{j \in [m]})$ to the **Receiver**.

4 Random Vector Oblivious Linear Evaluation

In this section, we provide a full description and proof for the oblivious linear evaluation sub-protocol. We first provide the functionality in Section 4.1.

4.1 Functionality

Looking ahead, our functionality is of a random VOLE where the inputs of both the **Sender** and **Receiver** are chosen uniformly (as opposed to [DKLs24], in which the inputs of the **Sender** are not random). Moreover, as no intermediate outputs are needed in our two-party signing, the formulation of the functionality is slightly simpler compared to [DKLs24].

Functionality 4.1: $\mathcal{F}_{\text{RVOLE}}$: Random VOLE with delayed receiver's input

The functionality interacts with two parties, a **Sender** and a **Receiver**, and the ideal adversary $\mathcal{S}$ who can instruct the functionality to abort a session at any time. For concreteness, ℓ is the number of multiplications, and the multiplications are performed over Scalar .

Multiplication:

- If the **Sender** is honest and sends $(\text{multiply}, \text{sid})$ to the functionality, and if the **Receiver** is corrupted, sid is fresh, then notify the ideal adversary. Receive back $(\text{receiverInput}, \text{sid}, \beta, \mathbf{r})$ for $\beta \in \text{Scalar}$, $\mathbf{r} = (r_1, \dots, r_\ell) \in \text{Scalar}^\ell$. Sample $\boldsymbol{\alpha} = (\alpha_1, \dots, \alpha_\ell) \in \text{Scalar}^\ell$, and compute for every $i \in [\ell]$: $t_i = \alpha_i \cdot \beta - r_i$. Send $(\text{senderOutput}, \text{sid}, \boldsymbol{\alpha}, \mathbf{t})$ to the **Sender** for $\mathbf{t} = (t_1, \dots, t_\ell)$.
 - If the **Receiver** is honest and sends $(\text{multiply}, \text{sid})$ to the functionality, and the **Sender** is corrupted, sid is fresh, then notify the ideal adversary. Receive back $(\text{senderInput}, \text{sid}, \boldsymbol{\alpha}, \mathbf{t})$ for $\boldsymbol{\alpha} = (\alpha_1, \dots, \alpha_\ell)$, $\mathbf{t} = (t_1, \dots, t_\ell)$ both in Scalar^ℓ . Sample $\beta \in \text{Scalar}$ and set for every $i \in [\ell]$, compute $r_i = \alpha_i \cdot \beta - t_i$. Send $(\text{receiverOutput}, \text{sid}, \beta, \mathbf{r})$ to the **Receiver**.
 - If both parties are honest and send $(\text{multiply}, \text{sid})$ to the functionality, then sample $\boldsymbol{\alpha} \in \text{Scalar}^\ell$, $\beta \in \text{Scalar}$. Sample $\mathbf{t} = (t_1, \dots, t_\ell) \in \text{Scalar}^\ell$, and set for every $i \in [\ell]$ the value $r_i = \alpha_i \cdot \beta - t_i$. Send $(\text{senderOutput}, \text{sid}, \boldsymbol{\alpha}, \mathbf{t})$ to the **Sender** and $(\text{receiverOutput}, \text{sid}, \beta, \mathbf{r})$ to the **Receiver**.
-

4.2 The Protocol

We provide three different variants for the protocol, where they are numbered with an increasing number of OTs needed. Nevertheless, as we focus on the two-party setting, and advocate using Variant II, we describe it first to ease translation into an implementation. We recall all three variants:

1. Variant I (Section 4.3): Using additional rounds, and picking $\mathbf{g}$ at the end. This reduces the overall number of OTs, as $\mathbf{g}$ is chosen uniformly at random at the end of the protocol.
2. Variant II (Section 4.2.1): This is essentially Variant I, after applying a round compressing transformation, and $\mathbf{g}$ is chosen by applying the random oracle on the transcript of the protocol.
3. Variant III (Section 4.4): $\mathbf{g}$ is chosen at the onset of the computation, and therefore requires the most number of OTs. This is the original protocol by [DKLs24].

We also remark that a “tighter” functionality can be described for each variant. For example, in Variant III, the **Receiver**’s input β is known already at the end of the first round, and therefore one can describe a functionality which gives first β to the **Receiver**, and later its additive share. However, for simplicity, we provide a single functionality, which is a common ground for all three variants.

4.2.1 Variant II: Full Description

We describe the protocol of [DKLs24] while deferring the choice of the “gadget vector” ($\mathbf{g}$) to the end of the protocol instead of the beginning. We write the protocol in an “implementation-friendly” description, clearly identifying the input-output of each next-message function.

Protocol 4.2: VOLE Protocol in the $\mathcal{F}_{\text{ROM}}$ - $\mathcal{F}_{\text{RSOT}}$ -hybrid model

The protocol:

1. **Receiver Round 1** VOLE.R.Round1(sid)
 Upon receiving (multiply, sid) from the environment:
 - (a) The **Receiver** chooses a random $\mathbf{w} \leftarrow \{0, 1\}^m$, and calls (receiverInput, sid||RSOT, $\mathbf{w}$) to $\mathcal{F}_{m \times \text{RSOT}}^{\text{Scalar}^{\ell+1}}$ (Functionality 3.4).
-

2. **Sender Round 2:** $((\alpha_1, \dots, \alpha_\ell), (t_1, \dots, t_\ell)) \leftarrow \text{VOLE.S.Round2}(\text{sid}, \mathbf{T}^0, \mathbf{T}^1)$
 Upon receiving (multiply, sid) from the environment,
 and upon receiving (senderOutput, sid||RSOT, $\mathbf{T}^0, \mathbf{T}^1$) from $\mathcal{F}_{m \times \text{RSOT}}^{\text{Scalar}^{\ell+1}}$ (Functionality 3.4):
 - (a) The **Sender** samples $\alpha_1, \dots, \alpha_\ell, \hat{\alpha}$, each in **Scalar**, and sets

$$\boldsymbol{\alpha} = (\alpha_1, \dots, \alpha_\ell, \hat{\alpha}) \in \text{Scalar}^{\ell+1},$$

and

$$\mathbf{A} = \begin{pmatrix} \boldsymbol{\alpha} \\ \dots \\ \boldsymbol{\alpha} \end{pmatrix} \in \text{Scalar}^{m \times (\ell+1)}.$$

i.e., $\mathbf{A}$ is a $m \times (\ell + 1)$ matrix, where all the rows are the same $-\boldsymbol{\alpha}$.

- (b) Define:

$$\mathbf{Q} = \mathbf{T}^0 - \mathbf{T}^1 + \mathbf{A} \in \text{Scalar}^{m \times (\ell+1)},$$

and denote:

$$\mathbf{T} = \mathbf{T}^0 = \begin{pmatrix} t_{1,1} & \dots & t_{1,\ell} & \hat{t}_1 \\ t_{2,1} & \dots & t_{2,\ell} & \hat{t}_2 \\ \dots & & & \\ t_{m,1} & \dots & t_{m,\ell} & \hat{t}_m \end{pmatrix}$$

- (c) The **Sender** computes $(\chi_1, \dots, \chi_\ell) \leftarrow \mathcal{F}_{\text{ROM}}^{\text{Scalar}^\ell}(\text{sid}, \text{chi}, \mathbf{Q})$.²

- (d) We write $\boldsymbol{\chi} = (\chi_1, \dots, \chi_\ell, 1)$.

²In the plain model (after instantiating $\mathcal{F}_{\text{RSOT}}$) - the input to the $\mathcal{F}_{\text{ROM}}$ should also include the transcript of the oblivious transfer.

(e) The **Sender** computes:

$$\mathbf{u} = \mathbf{T} \cdot \boldsymbol{\chi} = \begin{pmatrix} \chi_1 \cdot t_{1,1} + \dots + \chi_\ell \cdot t_{1,\ell} + \hat{t}_1 \\ \dots \\ \chi_1 \cdot t_{m,1} + \dots + \chi_\ell \cdot t_{m,\ell} + \hat{t}_m \end{pmatrix},$$

and

$$v = \langle \boldsymbol{\alpha}, \boldsymbol{\chi} \rangle = \chi_1 \cdot \alpha_1 + \dots + \chi_\ell \cdot \alpha_\ell + \hat{\alpha}.$$

(f) Compute $H_u = \mathcal{F}_{\text{ROM}}^{2\kappa}(\text{sid}, \mathbf{u}, \mathbf{u})$.

(g) The **Sender** defines $m_2 = (\mathbf{Q}, H_u, v)$.

(h) The **Sender** defines $\mathbf{g} = \mathcal{F}_{\text{ROM}}^{\text{Scalar}^m}(\text{sid}, \text{gadget}, m_2)$. See Footnote 2.

(i) **Outgoing Message:** The sender sends to the **Receiver** the message $m_2 = (\mathbf{Q}, H_u, v)$.

(j) **Computing Output:** The **Sender** outputs $(\alpha_1, \dots, \alpha_\ell), (t_1, \dots, t_\ell)$, where each t_i is defined by:

$$t_i = - \sum_{j=1}^m g_j \cdot t_{j,i}.$$

3. **Receiver Computes Output:** $((r_1, \dots, r_\ell), b) \leftarrow \text{VOLE.R.ComputeOutput}(m_2, \mathbf{S})$
 Upon receiving $m_2 = (\mathbf{Q}, H_u, v)$ from the **Sender**, and $(\text{receiverOutput}, \text{sid} \parallel \text{RSOT}, \mathbf{S})$ from $\mathcal{F}_{m \times \text{RSOT}_{\text{Scalar}^{\ell+1}}}$:

(a) Compute a matrix $\mathbf{R} \in \text{Scalar}^{m \times (\ell+1)}$ as follows. For every $j \in [m], i \in [\ell+1]$:

$$\mathbf{R}_{j,i} = \mathbf{S}_{j,i} + w_j \cdot \mathbf{Q}_{j,i},$$

and write:

$$\mathbf{R} = \begin{pmatrix} r_{1,1} & \dots & r_{1,\ell} & \hat{r}_1 \\ r_{2,1} & \dots & r_{2,\ell} & \hat{r}_2 \\ \dots & \dots & \dots & \dots \\ r_{m,1} & \dots & r_{m,\ell} & \hat{r}_m \end{pmatrix}.$$

Note that for every $j \in [m], i \in [\ell]$, it is supposed to hold that:

$$r_{j,i} - t_{j,i} = w_j \cdot \alpha_i, \quad \hat{r}_j - \hat{t}_j = w_j \cdot \hat{\alpha}. \quad (1)$$

(b) Invoke $(\chi_1, \dots, \chi_\ell) \leftarrow \mathcal{F}_{\text{ROM}}^{\text{Scalar}^\ell}(\text{sid}, \text{chi}, \mathbf{Q})$. See Footnote 2.

(c) Let $\boldsymbol{\chi} = (\chi_1, \dots, \chi_\ell, 1)$.

(d) For every $j \in [m]$, compute:

$$u'_j = \langle \mathbf{R}_j, \boldsymbol{\chi} \rangle - w_j \cdot v$$

and let $\mathbf{u}' = (u'_1, \dots, u'_m)$. Then, verify that

$$H_u = \mathcal{F}_{\text{ROM}}^{2\kappa}(\text{sid}, \mathbf{u}, \mathbf{u}').$$

If not, then it aborts.

(e) The **Receiver** defines $\mathbf{g} = \mathcal{F}_{\text{ROM}}^{\text{Scalar}^m}(\text{sid}, \text{gadget}, m_2)$. See Footnote 2.

(f) The **Receiver** computes $b = \langle \mathbf{g}, \mathbf{w} \rangle$.

(g) **Output:** The **Receiver** outputs b and $(r_1, \dots, r_\ell)$, each in **Scalar**, such that:

$$r_i = \sum_{j=1}^m g_j \cdot r_{j,i} .$$

4.3 Variant I: Simpler Protocol – More Rounds

Protocol 4.2 uses “Fiat-Shamir-like” transformation in two steps to compress the number of rounds. To simplify the analysis, we first analyze the protocol using additional rounds; we then apply the “Fiat-Shamir-like” transformation to obtain the real protocol, while reducing its security to the security of our additional-rounds protocol. Concretely, the full protocol is as follows:

1. **Round 1 – Receiver:** The **Receiver** runs Step 1 in its entirety; in particular, it sends $(\text{receiverInput}, \text{sid} \parallel \text{RSOT}, \mathbf{w})$ to $\mathcal{F}_{m \times \text{RSOT}_{\text{Scalar}^{\ell+1}}}$.
2. **Round 2 – Sender:** The **Sender** runs Step 2 until Step 2c, and it sends the message $\mathbf{Q}$ to **Receiver**.
3. **Round 3 – Receiver:** Instead of deriving χ from the random oracle as in the real protocol as in Step 2c, the receiver chooses it. That is, the receiver chooses $(\chi_1, \dots, \chi_\ell)$ uniformly at random in Scalar^ℓ , and sends it to the **Sender**.
4. **Round 4 – Sender:** The **Sender** continues to run Step 2, from Step 2e to Step 2g. It then sends $(\mathbf{u}, v)$ to the **Receiver**. For simplicity, we send $\mathbf{u}$ in its entirety instead of $H_u = \mathcal{F}_{\text{ROM}}^{2\kappa}(\text{sid}, \mathbf{u}, \mathbf{u})$.
5. **Round 5 – Receiver:** Instead of deriving $\mathbf{g}$ from the random oracle as in Step 2h, the receiver samples $\mathbf{g}$ uniformly at random in Scalar^m and sends it to the **Sender** after the verification. Moreover, instead of querying the random oracle on $\mathbf{u}'$, it verifies directly that $\mathbf{u}' = \mathbf{u}$ where $\mathbf{u}$ was received from the **Sender**. If the verification passes, it computes its output $b = \langle \mathbf{g}, \mathbf{w} \rangle$ and $(r_1, \dots, r_\ell)$ as in Step 3.
6. **The Sender computes its output:** Similarly to Step 2j – given $\mathbf{g}$ the **Sender** can now compute its output $(t_1, \dots, t_\ell)$.

4.4 Variant III: The Protocol of [DKLs24]

Protocol 4.2 is a small modification of the protocol described in [DKLs24], where the modifications are:

1. **Sender Random OT instead of Endemic OT:** We work in a different hybrid model than that of [DKLs24], which also implies different properties from the underlying OT. We review the importance of this modification in Section 4.5.
2. **Defer the choice of the gadget vector:** The gadget vector $\mathbf{g}$ in [DKLs24] is chosen at the onset of the computation, as a function of the **sid** only. As such, the **Receiver** can obtain its value $b = \langle \mathbf{g}, \mathbf{w} \rangle$ at the end of the first round (and also output it as an intermediate output). In contrast, we defer the choice of $\mathbf{g}$ to the end of the protocol. This makes b available to the **Receiver** only at the end of round 2, but reduces the number of OTs, as we will see in the analysis.

3. **Number of masking elements:** The construction [DKLs24] adds ρ elements to each row (i.e., $\mathbf{A} \in \text{Scalar}^{m \times (\ell + \rho)}$), where those elements mask the information revealed when multiplying the rows with χ . We describe the protocol with just $\rho = 1$, which suffices for Secp256k1, and for most applications. In fact, the description in [DKLs24] requires that $\rho = \lceil \log q / \kappa \rceil$ with κ as the computational security parameter (and recall that $\log q$ is the number of bits for representing an element in the domain of the multiplication). For Secp256k1 ($\log q = 256$) and $\kappa = 128$, this implies that $\rho = \lceil 256 / 128 \rceil = 2$ elements have to be added. This looks like a typo, and $\rho = \lceil \kappa / \log q \rceil = 1$ suffices. As we will see in the analysis, $\rho = 1$ suffices as long as $\log q \geq \kappa + 2 \cdot \log m$, see Claim B.8. We presented the protocol when ρ is fixed to one, and we just add this requirement to the theorem statement.
4. **Fully-random VOLE:** In the protocol of [DKLs24], the functionality is slightly different: The **Sender** inputs particular inputs $(a_1, \dots, a_\ell)$, the random “input” b of the **Receiver** is chosen by the protocol, and the parties compute additive shares of $(a_1 \cdot b, \dots, a_\ell \cdot b)$. This is done by letting the **Sender** send $a_i - \alpha_i$ in the last round, and then the **Receiver** adds to each of its output shares the value $(a_i - \alpha_i) \cdot b$. We use fully random VOLE, and delay sending this message to the signing phase. This adds a round to the final protocol, but enables the separation between a truly input-independent preprocessing phase and an extremely fast online phase.

4.5 Attack on the Protocol in the Endemic OT-Hybrid Model

The VOLE protocol of [DKLs24] is proven in the Endemic OT [MR19]-hybrid model, which allows a malicious **Sender** to pick its inputs to the functionality. We show that the VOLE is insecure in this hybrid model. We first describe the functionality:

Functionality 4.3: $\mathcal{F}_{m \times \text{Endemic}_D}$ – Endemic OT

The functionality interacts with the **Sender** and the **Receiver**, and with an ideal adversary $\mathcal{S}$ who can instruct to abort at any time. The domain is $\mathcal{D}$.

1. If the **Sender** is corrupted, upon receiving $(\text{senderInput}, \text{sid}, \mathbf{T}^0, \mathbf{T}^1)$ for a fresh sid and $\mathbf{T}^0, \mathbf{T}^1 \in \mathcal{D}^m$, then sample $\mathbf{w} \in \{0, 1\}^m$.
2. If the **Receiver** is corrupted, then upon receiving $(\text{receiverInput}, \text{sid}, \mathbf{w})$ from the adversary, sample $\mathbf{T}^0, \mathbf{T}^1 \in \mathcal{D}^m$.
3. If both parties are honest, then upon receiving both $(\text{senderInput}, \text{sid})$ and $(\text{receiverInput}, \text{sid})$ for the same fresh sid , then sample $\mathbf{w} \in \{0, 1\}^m$ and $\mathbf{T}^0, \mathbf{T}^1 \in \mathcal{D}^m$.

For every $i \in [m]$ set $\mathbf{S}_i = (1 - w_i) \cdot \mathbf{T}_i^0 + w_i \cdot \mathbf{T}_i^1 = \mathbf{T}_i^0 + w_i \cdot (\mathbf{T}_i^1 - \mathbf{T}_i^0)$. Send $(\text{senderOutput}, \text{sid}, \mathbf{T}^0, \mathbf{T}^1)$ to the **Sender**, and $(\text{receiverOutput}, \text{sid}, \mathbf{w}, \mathbf{S})$ to the **Receiver**.

The following refers to variants II and III of the protocol, when working with the endemic OT-hybrid model. The purpose of the protocol’s consistency check is to ensure that the **Sender** indeed places the same row everywhere in the matrix $\mathbf{A}$. To verify this, a challenge vector $\chi = (\chi_1, \dots, \chi_\ell, 1)$ is sampled, and the **Sender** computes $\mathbf{u} = \mathbf{T}^0 \cdot \chi$ and $v = \langle \mathbf{A}_1, \chi \rangle$. For security, it is crucial that whenever two rows differ, $\mathbf{A}_i \neq \mathbf{A}_j$, we have $\langle \mathbf{A}_i, \chi \rangle \neq \langle \mathbf{A}_j, \chi \rangle$. In other words, χ acts as a universal hash function: for every $\mathbf{A}_i \neq \mathbf{A}_j$, the probability over random χ that $\langle \mathbf{A}_i, \chi \rangle = \langle \mathbf{A}_j, \chi \rangle$ is negligible. See Claim B.7 in the analysis.

In [DKLs24], the challenge χ is derived via the random oracle by hashing $\mathbf{Q}$ (together with sid , domain separators, etc.), but without incorporating anything that binds the OT phase (see Step 2c in the description of Protocol 4.2). In the OT-hybrid model, no transcript binds the OT phase, since the parties merely send inputs and receive outputs from the functionality. Moreover, they cannot hash their OT inputs or outputs, since the two parties do not observe the same values. In the endemic OT-hybrid model, where a corrupted sender may freely choose its OT inputs, this opens the door for the following attack, which completely nullifies the verification step:

- The corrupted **Sender** first samples $\mathbf{Q} \in \text{Scalar}^{m \times (\ell+1)}$ uniformly at random.
- It derives χ as a function of $\mathbf{Q}$ using the random oracle; concretely, it computes $(\chi_1, \dots, \chi_\ell) = \mathcal{F}_{\text{ROM}}^{\text{Scalar}^\ell}(\text{sid}, \text{chi}, \mathbf{Q})$ and sets $\chi = (\chi_1, \dots, \chi_\ell, 1)$.
- It then chooses any matrix $\mathbf{A}$ it wishes, with arbitrarily inconsistent rows, leaving only the last column undetermined.
- It samples $v \leftarrow \text{Scalar}$ uniformly at random.
- For every $j \in [m]$, it sets the last coordinate of row j of $\mathbf{A}$ to

$$\hat{\alpha}_j = v - \chi_1 \cdot \alpha_{j,1} - \dots - \chi_\ell \cdot \alpha_{j,\ell}.$$

Note that for every $i, j \in [m]$ it holds that

$$\langle \mathbf{A}_j, \chi \rangle = v = \langle \mathbf{A}_i, \chi \rangle,$$

so the universal-hash test is perfectly neutralized.

- Since this is the endemic OT hybrid model, the **Sender** can pick its own inputs to the OT. The adversary samples $\mathbf{T}^0$ uniformly and sets $\mathbf{T}^1 = \mathbf{T}^0 + \mathbf{A} - \mathbf{Q}$.
- In the endemic OT phase, it sends $(\text{senderInput}, \text{sid}, \mathbf{T}^0, \mathbf{T}^1)$ to the endemic-OT functionality. It computes $\mathbf{u} = \langle \mathbf{T}^0, \chi \rangle$ honestly and sets $H_u = \mathcal{F}_{\text{ROM}}^{2\kappa}(\text{sid}, \mathbf{u}, \mathbf{u})$. It then sends $(\mathbf{Q}, H_u, v)$ to the honest **Receiver**.
- The honest **Receiver** recovers $\mathbf{R}$, derives the same χ , and for every j computes

$$\begin{aligned} u'_j &= \langle \mathbf{R}_j, \chi \rangle - w_j \cdot v \\ &= \langle \mathbf{S}_j + w_j \cdot \mathbf{Q}_j, \chi \rangle - w_j \cdot v \\ &= \langle \mathbf{T}_j^0 + w_j \cdot \mathbf{A}_j, \chi \rangle - w_j \cdot v \\ &= \langle \mathbf{T}_j^0, \chi \rangle + w_j \langle \mathbf{A}_j, \chi \rangle - w_j \cdot v \\ &= u_j + w_j v - w_j \cdot v \\ &= u_j, \end{aligned}$$

so the verification step always passes, although the rows are not the same.

- This breaks the correctness property of the protocol.

How to prevent this attack. We describe two complementary countermeasures.

Include the OT transcript when deriving χ . When applying a Fiat-Shamir-like transformation to a simpler interactive protocol, it is standard to hash the entire transcript up to that point. In the OT-hybrid model, there is no such transcript, but in a real protocol the OT execution *does* generate one. This is helpful if the transcript of the OT protocol commits the sender to its

input, and the concrete OT protocol that is used has to satisfy that property. Incorporating this transcript into the hash that produces χ forces χ to depend on the OT inputs.

Use Sender Random OT. We prove that the protocol is secure when instantiated with the Sender Random OT functionality (see Functionality 3.4). This functionality prevents a corrupted sender from choosing its OT inputs. In Sender Random OT, the matrices $\mathbf{T}^0$ and $\mathbf{T}^1$ are generated by the functionality, and the sender has no control over them. Since $\mathbf{Q}$ and $\mathbf{A}$ (given $\mathbf{T}^0, \mathbf{T}^1$) are in one-to-one correspondence, any choice of $\mathbf{A}$ uniquely determines $\mathbf{Q}$, which in turn uniquely determines χ . We show that it is infeasible to find a “bad” pair $(\mathbf{Q}, \chi)$ enabling the attack above; with overwhelming probability, if $\mathbf{A}_i \neq \mathbf{A}_j$ then $\langle \mathbf{A}_i, \chi \rangle \neq \langle \mathbf{A}_j, \chi \rangle$. We show that the protocol is secure in this hybrid model, while applying the random oracle only on $\mathbf{Q}$ and not on the OT-transcript. Nevertheless, in implementations we recommend also including the OT-transcript.

4.6 Analysis

In Appendix B we provide a full proof for the following theorem:

Theorem 4.4. *Assuming that $\log q \geq \kappa + 2 \log m$, and in addition the following constraint on m , the respective protocol securely realizes Functionality 4.1 in the $\mathcal{F}_{m \times \text{RSOT}}^{\text{Scalar}^{\ell+1}}$ - $\mathcal{F}_{\text{ROM}}$ -hybrid model.*

Protocol	Number of OTs (m)	See
Variant I (Section 4.3)	$m \geq \log q + 3 \cdot s$	Theorem B.1
Variant II (Section 4.2.1)	$m \geq \log q + 2 \cdot \kappa + 3 \cdot s$	Theorem B.16
Variant III (Section 4.4)	$m \geq \log q + 7 \cdot s + 2 \log(s + 1) + 2s \cdot \log(e \cdot m/s)$	Theorem B.17

5 Applying Leftover Hash Lemma: Various Games and their Required Parameters

The three variants in Section 4 refer to three different security games that use the leftover-hash lemma [ILL89], which results in different number of OTs, m . In this section, we formalize the three different security games. First, recall that the leftover hash lemma compares the uniform distribution to a random choice of a hash function from a universal hash, followed by an output of that hash on a random input. We start with the definition of a universal hash.

Definition 5.1. *Let $\mathcal{H}$ be a family of hash functions from $\mathcal{X} \rightarrow \mathcal{Y}$. $\mathcal{H}$ is a universal family if for every $x_1, x_2 \in \mathcal{X}$: $\Pr[h(x_1) = h(x_2)] \leq 1/|\mathcal{Y}|$, where h is chosen uniformly from $\mathcal{H}$.*

Recall that the min-entropy of a random variable X is $-\log(\max_x p(x))$.

5.1 Variant I: Plain Leftover Hash Lemma

Lemma 5.2 ([ILL89]). *Let $\mathcal{H}$ be a universal hash function family. Let X be any probability distribution over $\mathcal{X}$ that has min entropy δ . Let H be the uniform distribution over $\mathcal{H}$, and let $U_{\mathcal{Y}}$ be the uniform distribution over $\mathcal{Y}$. Then:*

$$\text{SD}((H, H(X)), (H, U_{\mathcal{Y}})) \leq \frac{1}{2} \sqrt{\frac{|\mathcal{Y}|}{2^\delta}}$$

Henceforth, we denote

$$\epsilon_{\text{LHL}} := \frac{1}{2} \sqrt{|\mathcal{Y}|/2^\delta} \quad (2)$$

Concrete instantiation in our setting. We consider Variant I of the protocol where the **Receiver** chooses $\mathbf{g}$ at the end of the protocol, and sends it to the **Sender**.

The proof assumes that the corrupted **Sender** sends messages that may be malformed, i.e., it may cheat in some of them. In the analysis, we can delay the choice of the **Receiver**'s input $\mathbf{w}$ until after the **Sender**'s messages are fixed. For a given transcript sent by the **Sender**, some choices of $\mathbf{w}$ will cause the **Receiver** to detect cheating, while others will not. Hence, the messages sent by the corrupted **Sender** implicitly induce a set of constraints on $\mathbf{w}$: if the verification succeeds and the **Sender** is not caught cheating, then $\mathbf{w}$ must satisfy these constraints. In the final step of the proof, we show that under these constraints, the value $b = \mathbf{g}(\mathbf{w}) = \langle \mathbf{g}, \mathbf{w} \rangle$ remains close to uniform.

For a set of constraints $\mathbf{c} = \text{constraints} = \{(j, \rho) \in [m] \times \{0, 1\}\}$, let $W_{\mathbf{c}}$ be the uniform distribution over $\{0, 1\}^m$ that is consistent with $\mathbf{c}$, i.e., for every $(j, \rho) \in \mathbf{c}$ we fix $w_j = \rho$ and set all the other indices uniformly at random. Thus, $W_{\mathbf{c}}$ has a min-entropy of $m - |\mathbf{c}|$. We have the following:

- The corrupted **Sender** might send messages that implicitly fixes some set of constraints $\mathbf{c} = \{(j, \rho) \in [m] \times \{0, 1\}\}$, such that $|\mathbf{c}| \leq s$. For each such constraint, the **Sender** takes a risk of getting caught with probability $1/2$, and therefore it suffices to bound the number of constraints by s .
- Let $\mathcal{W}_{\mathbf{c}}$ to be the set of all binary strings of length m that are consistent with $\mathbf{c}$. That is, for every (j, ρ) and for every $\mathbf{w} \in \mathcal{W}_{\mathbf{c}}$ it holds that $w_j = \rho$.
- Let $W_{\mathbf{c}}$ be the uniform distribution over $\mathcal{W}_{\mathbf{c}}$. Thus, $W_{\mathbf{c}}$ has min entropy $m - |\mathbf{c}| \geq m - s$.
- The set $\mathbf{g} = (g_1, \dots, g_m)$ defines a hash function over **Scalar**, where the input comes from $\mathcal{W}_{\mathbf{c}}$: given $\mathbf{w}$ that is consistent with $\mathbf{c}$, the hash is defined as $\mathbf{g}(\mathbf{w}) = \langle \mathbf{g}, \mathbf{w} \rangle$. We show that this is a universal hash function. For every fixed set of constraints $\mathbf{c}$ with $|\mathbf{c}| \leq s$, and for every $\mathbf{w}, \mathbf{w}' \in \mathcal{W}_{\mathbf{c}}$ with $\mathbf{w} \neq \mathbf{w}'$, there must exist an index j for which $w_j \neq w'_j$. Without loss of generality, assume that $w_j - w'_j = 1$. Then:

$$\Pr [\langle \mathbf{g}, \mathbf{w} \rangle = \langle \mathbf{g}, \mathbf{w}' \rangle] = \Pr \left[g_j = - \sum_{k \neq j} (w'_k - w_k) \cdot g_k \right] = \frac{1}{|\text{Scalar}|}.$$

Putting it all together, we get that

Claim 5.3. *For every fixed set of constraints $\mathbf{c} \subseteq [m] \times \{0, 1\}$ of cardinality at most s , it holds that:*

$$\text{SD}((\mathbf{G}, \mathbf{G}(W_{\mathbf{c}})), (\mathbf{G}, U_{\text{Scalar}})) \leq \frac{1}{2} \sqrt{\frac{|\text{Scalar}|}{2^{m-s}}},$$

This is bounded by 2^{-s} when $m \geq \log q + 3 \cdot s$.

5.2 Variant II: Leftover Hash Lemma with Adversarially-Limitedly-Chosen Hash

We now explore Variant II: instead of choosing the hash $\mathbf{g}$ uniformly at random by the **Receiver** at the end of the protocol, $\mathbf{g}$ is chosen by the random oracle. That is, $\mathbf{g}$ is a result of the random

oracle applied on the message that the corrupted **Sender** is about to send. This gives the adversary an additional capability: It can try different messages and see their corresponding $\mathbf{g}$ (by querying the random oracle). That is, it can try different messages before sending its actual message.

We model it as an adversary that can provide multiple sources, all of which have high-min-entropy, and for each source an independent hash is chosen. Then, after seeing the chosen hash, the adversary can pick which source to proceed with. Taking a worst-case adversary, we can assume it proceeds with the “weakest” hash, i.e., the hash for which $\text{SD}(h_i(X_i), U_Y)$ is the largest. We have the following two distributions:

Dist_{1,Q}:

- Let $X_1, \dots, X_Q$ be any probability distributions over $\mathcal{X}$ with min entropy δ each.
- Choose $h_1, \dots, h_Q$ uniformly and independently at random from $\mathcal{H}$.
- Let $i = \text{argmax}(\text{SD}(h_i(X_i), U_Y))$.
- Output: $(h_i, h_i(X_i))$.

Dist_{2,Q}:

- Let $X_1, \dots, X_Q$ be any probability distributions over $\mathcal{X}$ with min entropy δ each.
- Choose $h_1, \dots, h_Q$ uniformly and independently at random from $\mathcal{H}$.
- Let $i = \text{argmax}(\text{SD}(h_i(X_i), U_Y))$.
- Output: (h_i, U_Y) .

Lemma 5.4. *For every $0 < \epsilon < 1$ and $Q > 1$ it holds that:*

$$\text{SD}(\text{Dist}_{1,Q}, \text{Dist}_{2,Q}) \leq Q \cdot \frac{1}{2} \sqrt{|\mathcal{Y}|/2^\delta} \cdot \epsilon^{-1} + \epsilon$$

Proof. For a fixed probability distribution X_i and a fixed hash h , define:

$$\Delta_{h,X_i} = \text{SD}(h(X_i), U_Y) .$$

We now consider the expected value of Δ_{h_i,X_i} for a random h_i (and the probability distribution X_i is fixed). We have:

$$\begin{aligned} E_{h_i \leftarrow H}(\Delta_{h_i,X_i}) &= \sum_{h \in \mathcal{H}} \Pr[H = h] \cdot \Delta_{h,X_i} \\ &= \sum_{h \in \mathcal{H}} \Pr[H = h] \cdot \left(\frac{1}{2} \cdot \sum_{y \in \mathcal{Y}} |\Pr[h(X_i) = y] - \Pr[U_Y = y]| \right) \\ &= \frac{1}{2} \sum_{h \in \mathcal{H}} \sum_{y \in \mathcal{Y}} |\Pr[(H, H(X_i)) = (h, y)] - \Pr[(H, U_Y) = (h, y)]| \\ &= \text{SD}((H, H(X_i)), (H, U_Y)) \leq \epsilon_{\text{LHL}} . \end{aligned}$$

Using Markov’s inequality, for every $\epsilon \geq 0$ it holds that:

$$\Pr_{h_i \leftarrow H}[\Delta_{h_i,X_i} \geq \epsilon] \leq E(\Delta_{h_i,X_i}) \cdot \epsilon^{-1} \leq \epsilon_{\text{LHL}} \cdot \epsilon^{-1} .$$

Let **Bad** be the event (either in $\text{Dist}_{1,Q}$ or $\text{Dist}_{2,Q}$) such that, for each X_i (for $i \in [Q]$) we sample h_i uniformly and independently at random and it holds that $\Delta_{h_i,X_i} \geq \epsilon$. By a union bound over all Q pairs (h_i, X_i) , we get that:

$$\Pr[\text{Bad}] = \Pr[\exists i \in [Q] \quad \Delta_{h_i,X_i} \geq \epsilon] \leq Q \cdot \epsilon_{\text{LHL}} \cdot \epsilon^{-1} .$$

We have:

1. In both experiments the probability of **Bad** is bounded by $Q \cdot \epsilon_{\text{LHL}} \cdot \epsilon^{-1}$.
2. The statistical distance between the two experiments when **Bad** does not occur is at most ϵ , as it does not matter which i is chosen.

We have that (and by applying Eq. (2) this completes the proof):

$$\begin{aligned}
& \text{SD}(\text{Dist}_{1,Q}, \text{Dist}_{2,Q}) \\
&= \text{SD}(\text{Dist}_{1,Q} \mid \text{Bad}, \text{Dist}_{2,Q} \mid \text{Bad}) \cdot \Pr[\text{Bad}] \\
&\quad + \text{SD}(\text{Dist}_{1,Q} \mid \overline{\text{Bad}}, \text{Dist}_{2,Q} \mid \overline{\text{Bad}}) \cdot \Pr[\overline{\text{Bad}}] \\
&\leq Q \cdot \epsilon_{\text{LHL}} \cdot \epsilon^{-1} + \epsilon.
\end{aligned}$$

□

Instantiation in Our Setting This variant corresponds to Variant II, and the exact description of Protocol 4.2. We have the following claim:

Claim 5.5. *Let $m \geq \log q + 3 \cdot s + 2 \cdot \kappa$. For the following two distributions, it holds that:*

$$\text{SD}(\mathbf{Dist}_{1,Q}, \mathbf{Dist}_{2,Q}) \leq Q \cdot 2^{-\kappa} + 2^{-s},$$

when the distributions are defined as:

Dist_{1,Q}:

- Fix constraints $\mathbf{c}_1, \dots, \mathbf{c}_Q$ and let $W_{\mathbf{c}_1}, \dots, W_{\mathbf{c}_Q}$ be the corresponding uniform distributions conditioned on being consistent with the corresponding constraints.
- Choose $\mathbf{g}_1, \dots, \mathbf{g}_Q$ uniformly and independently at random from Scalar^m .
- Let $i = \arg\max(\text{SD}(\mathbf{g}_i(W_{\mathbf{c}_i}), U_{\text{Scalar}}))$.
- Output: $(\mathbf{g}_i, \mathbf{g}_i(W_{\mathbf{c}_i}))$.

Dist_{2,Q}:

- Fix constraints $\mathbf{c}_1, \dots, \mathbf{c}_Q$ and let $W_{\mathbf{c}_1}, \dots, W_{\mathbf{c}_Q}$ be the corresponding uniform distributions conditioned on being consistent with the corresponding constraints.
- Choose $\mathbf{g}_1, \dots, \mathbf{g}_Q$ uniformly and independently at random from Scalar^m .
- Let $i = \arg\max(\text{SD}(\mathbf{g}_i(W_{\mathbf{c}_i}), U_{\text{Scalar}}))$.
- Output: $(\mathbf{g}_i, U_{\text{Scalar}})$.

Proof. Using Lemma 5.4:

$$\text{SD}(\mathbf{Dist}_{1,Q}, \mathbf{Dist}_{2,Q}) \leq Q \cdot \epsilon_{\text{LHL}} \cdot \epsilon^{-1} + \epsilon.$$

When $\epsilon = 2^{-s}$, it suffices to require that $\epsilon_{\text{LHL}} \cdot \epsilon^{-1} \leq 2^{-\kappa}$, and so (recall Eq. (2)):

$$\begin{aligned}
\epsilon_{\text{LHL}} \cdot \epsilon^{-1} &\leq 2^{-\kappa} \\
\frac{1}{2} \sqrt{\frac{|\text{Scalar}|}{2^{m-s}}} \cdot 2^s &\leq 2^{-\kappa}.
\end{aligned}$$

This implies that

$$2^{m-s} \geq \frac{1}{4} \cdot |\text{Scalar}| \cdot 2^{2(s+\kappa)}$$

and so we require that $m \geq \log q + 3 \cdot s + 2 \cdot \kappa$.

□

Concrete parameters. In our setting, we use the above Lemma with $\log q = 256$, $s = 60$ and $\kappa = 128$; This results in 692 OTs; we use 696 to obtain a multiple of 8.

5.3 Variant III: Leftover Hash Lemma with Adversarial Source-Fixing

This variant is the one needed in the original construction of [DKLs24] and corresponds to Variant III. In this variant, the hash is first chosen $h \leftarrow H$. Then, the adversary can pick the source X as a function of the hash function. For worst-case, we might assume that it picks the “worst” source for that function, i.e., the one for which $\text{SD}(h(X), U_Y)$ is maximal.

Dist_{1, $\tilde{X}$} :

- Choose h uniformly at random from $\mathcal{H}$.
- Let $\tilde{X}$ be the set of all probability distributions over $\mathcal{X}$ with min-entropy δ .
Let $X = \arg\max_{X \in \tilde{X}} (\text{SD}(h(X), U_Y))$.
- Sample x according to X and output $(h, h(x))$.

Dist_{2, $\tilde{X}$} :

- Choose h uniformly at random from $\mathcal{H}$.
- Sample y uniformly in $\mathcal{Y}$, and output (h, y) .

Lemma 5.6. *For every $0 < \epsilon < 1$ it holds that:*

$$\text{SD}(\text{Dist}_{1, \tilde{X}}, \text{Dist}_{2, \tilde{X}}) \leq |\tilde{X}| \cdot \frac{1}{2} \sqrt{|\mathcal{Y}|/2^\delta} \cdot \epsilon^{-1} + \epsilon.$$

Proof. Similarly to the proof of Lemma 5.4, we obtain that for a fixed source X and random h , for every $\epsilon \geq 0$ it holds that:

$$\Pr_{h \leftarrow H} [\Delta_{h,X} > \epsilon] \leq E(\Delta_{h,X}) \cdot \epsilon^{-1} \leq \epsilon_{\text{LHL}} \cdot \epsilon^{-1}.$$

This implies that for a random $h \leftarrow H$:

$$\Pr_{h \leftarrow H} [\exists X \in \tilde{X} \text{ s.t. } \Delta_{h,X} > \epsilon] \leq \sum_{X \in \tilde{X}} \Pr_h [\Delta_{h,X} > \epsilon] \leq |\tilde{X}| \cdot \epsilon_{\text{LHL}} \cdot \epsilon^{-1}.$$

Let **Bad** be the event (either in $\text{Dist}_{1, \tilde{X}}$ or $\text{Dist}_{2, \tilde{X}}$) such that a bad h is chosen, i.e., an h for which there exists some $X \in \tilde{X}$ such that $\Delta_{h,X} > \epsilon$. We have:

1. In both experiments the probability of **Bad** is bounded by $|\tilde{X}| \cdot \epsilon_{\text{LHL}} \cdot \epsilon^{-1}$.
2. The statistical distance between the two experiments conditioned on **Bad** does not occur is at most ϵ , as it does not matter which source is chosen.

We have that (and recall Eq. (2)):

$$\begin{aligned} & \text{SD}(\text{Dist}_{1, \tilde{X}}, \text{Dist}_{2, \tilde{X}}) \\ &= \text{SD}(\text{Dist}_{1, \tilde{X}} | \text{Bad}, \text{Dist}_{2, \tilde{X}} | \text{Bad}) \cdot \Pr[\text{Bad}] \\ & \quad + \text{SD}(\text{Dist}_{1, \tilde{X}} | \overline{\text{Bad}}, \text{Dist}_{2, \tilde{X}} | \overline{\text{Bad}}) \cdot \Pr[\overline{\text{Bad}}] \\ &\leq |\tilde{X}| \cdot \epsilon_{\text{LHL}} \cdot \epsilon^{-1} + \epsilon. \end{aligned}$$

□

5.3.1 Instantiation in Our Setting

In variant III – we wish to bound the statistical distance by 2^{-s} . This requires increasing m quite substantially.

Claim 5.7. *Assume that $m \geq \log q + 7s + 2 \log(s+1) + 2s \cdot \log \frac{e \cdot m}{s}$, and let $\mathcal{C}$ be the set of all possible constraints in $[m] \times \{0, 1\}$ of cardinality at most s . Then,*

$$\text{SD}(\mathbf{Dist}_{1,\mathcal{C}}, \mathbf{Dist}_2) \leq 2^{-s},$$

where $\mathbf{Dist}_{1,\mathcal{C}}, \mathbf{Dist}_2$ are defined as follows:

Dist_{1,C}:

- Choose $\mathbf{g}$ uniformly at random from Scalar^m .
- $\mathcal{C}$ is the set of all constraints of cardinality at most s .
- Let $\mathbf{c} = \arg\max_{\mathbf{c} \in \mathcal{C}} (\text{SD}(\mathbf{g}(W_{\mathbf{c}}), U_{\text{Scalar}}))$.
- Sample $\mathbf{w}$ according to $W_{\mathbf{c}}$ and output $(\mathbf{g}, \langle \mathbf{g}, \mathbf{w} \rangle)$.

Dist₂:

- Choose $\mathbf{g}$ uniformly at random from Scalar^m .
- Sample y uniformly in Scalar , and output $(\mathbf{g}, y)$.

Proof. We use Lemma 5.6, with $\tilde{X} = \mathcal{C}$. We set $\epsilon = 2^{-s}$, and it suffices to show that

$$|\mathcal{C}| \cdot \epsilon_{\text{LHL}} \cdot 2^s \leq 2^{-s},$$

i.e., we have to make sure that (recall Eq. (2):)

$$|\mathcal{C}| \cdot \frac{1}{2} \cdot \sqrt{\frac{|\text{Scalar}|}{2^{m-s}}} \cdot 2^s \leq 2^{-s},$$

In our setting, $\tilde{X}$ is the set of all constraints $\mathbf{c}$ with cardinality at most s . We can bound:

$$|\mathcal{C}| \leq \sum_{k=0}^s \binom{m}{k} 2^k \leq (s+1) \cdot \left(\frac{e \cdot m}{s}\right)^s \cdot 2^s,$$

i.e., we might consider between $k \in [0, \dots, s]$ constraints; each constraint involves choosing k locations and fixing k particular bits. Thus, we request that:

$$(s+1) \cdot \left(\frac{e \cdot m}{s}\right)^s \cdot 2^s \cdot \frac{1}{2} \cdot \sqrt{\frac{|\text{Scalar}|}{2^{m-s}}} \cdot 2^s \leq 2^{-s}$$

Taking log of both sides:

$$\log(s+1) + s \log(em/s) + s - 1 + \frac{1}{2}(\log q - (m-s)) + s \leq -s$$

Multiply through by 2:

$$2 \log(s+1) + 2s \log(em/s) + 4s - 2 + \log q - m + s \leq -2s$$

Rearrange for m , it suffices to require that:

$$m \geq \log q + 7s - 2 + 2 \log(s+1) + 2s \cdot \log \frac{e \cdot m}{s}$$

□

For $\log q = 256$ and $s = 60$, the minimum number of OTs needed for which the above holds is 1405 (in which case, the right-hand side is 1404.79).

6 The ECDSA Two-Party Signing Protocol

In this section, we present optimizations of [DKLs24] for the two-party setting. These include removing steps that are unnecessary in the two-party setting, and separating the protocol into an offline phase and an online phase.

6.1 The Functionality

We show that the protocol implements the following functionality, assuming that the inputs of the parties are valid and consistent [Lin24]:

1. The two parties hold the joint public keys (pk, pk_0, pk_1) such that $pk = pk_0 + pk_1$.
2. Each party P_i holds its secret key sk_i . It holds that $sk_i \cdot G = pk_i$ (and thus, $(sk_0 + sk_1) \cdot G = pk$).

The functionality is as follows:

Functionality 6.1: $\mathcal{F}_{\text{Sign}}^{\text{ECDSA}}$

- Upon receiving $(\text{Sign}, \text{sid}, m, pk, pk_0, pk_1; sk_i)$ from P_i for the two parties, verify that all public-inputs $(\text{Sign}, \text{sid}, m, pk, pk_0, pk_1)$ are valid and consistent as above ($pk = pk_0 + pk_1$, and $sk_i \cdot G = pk_i$). If not, then do nothing;
- Otherwise, set $sk = sk_0 + sk_1$, choose a random $r \leftarrow \text{Scalar}$ and compute $R = r \cdot G$, and let r^x be the x -coordinate of R . Compute

$$s = r^{-1} \cdot (H(m) + sk \cdot r^x) ,$$

and set $\sigma = (r^x, s)$. Send $(\text{sid}, pk, pk_0, pk_1, m, \sigma)$ to the ideal adversary. When the adversary sends OK, send the output to the honest party as well.

We remark that ECDSA restarts when $r^x = 0$, or $s = 0$; this happens by a negligible probability by the functionality, and we can assume that the functionality aborts if this is the case.

6.2 The Protocol

We provide a protocol that implements Functionality 6.1 assuming commitment (F_{com} , Functionality 3.3) and two instances of vector OLE ($\mathcal{F}_{\text{RVOLE}}$, Functionality 4.1), one instance in which P_0 is the sender and P_1 is the receiver, and one for the opposite direction. The protocol provides a few differences from that of [DKLs24]:

- **Random VOLE and pre-processing:** The most significant shift is the separation of the protocol into a **pre-processing phase** using Random VOLE (RVOLE) instead of standard VOLE. Whereas the protocol of [DKLs24] requires inputs (of the **Sender**) to be known at the time of the VOLE call, this protocol allows parties to generate random correlations $(\tilde{r}, \widetilde{sk}, \beta)$ ahead of time. During the signing phase, the parties simply “fix” these random values to their actual secrets using the offsets γ_{r_i} and γ_{sk_i} .

- **No corrections:** Since we deal with two parties, there is no need to correlate between the $n - 1$ different masking inputs of the **Receiver** to one global masking.
- **No $\mathcal{F}_{\text{zero}}$:** In the two-party setting, the $\mathcal{F}_{\text{zero}}$ functionality used in [DKLs24] to mask input shares of sk_i s unnecessary.

We proceed to the full protocol:

Protocol 6.2: Implementing Functionality 6.1 in the $(F_{\text{com}}, \mathcal{F}_{\text{RVOLE}})$ -hybrid model

Input pre-processing: A shared sid .

Hybrid model (sub-protocols): The parties run $\mathcal{F}_{\text{RVOLE}}$ twice, once when P_0 is the sender, and once when P_1 is the sender. The analysis is in the hybrid model, while the protocol here assumes the implementation given in Protocol 4.2, and follows its structure.

1. **First round P_i for $i \in \{0, 1\}$:**
 - (a) P_i samples a secret nonce $r_i \leftarrow \text{Scalar}$ and computes $R_i := r_i \cdot G$.
 - (b) P_i commits to R_i : it sends $(\text{commit}, \text{sid} \parallel \text{com}, i, R_i)$ to F_{com} (Functionality 3.3).
 - (c) Invoke $m_1^i = \text{VOLE.R.Round1}(\text{sid} \parallel \text{VOLE} \parallel 1 - i)$.
 - (d) **Message:** Send to P_{1-i} the message: m_1^i together with the commitment.
2. **Second round P_i for $i \in \{0, 1\}$:**
 - (a) Receive $(\text{receipt}, \text{sid} \parallel \text{com}, 1 - i)$ from F_{com} . If not, abort.
 - (b) P_i computes $(m_2^i, \tilde{r}_i, \tilde{\text{sk}}_i, \llbracket \tilde{r}_i \cdot \beta_{1-i} \rrbracket_i, \llbracket \tilde{\text{sk}}_i \cdot \beta_{1-i} \rrbracket_i) = \text{VOLE.S.Round2}(\text{sid} \parallel \text{VOLE} \parallel i, m_1^{1-i})$.
 - (c) **Message:** Send to P_{1-i} the message m_2^i .
3. **Compute Pre-processing:**
 - (a) P_i computes $(\beta_i, \llbracket \tilde{r}_{1-i} \cdot \beta_i \rrbracket_i, \llbracket \tilde{\text{sk}}_{1-i} \cdot \beta_i \rrbracket_i) = \text{VOLE.R.ComputeOutput}(\text{sid} \parallel \text{VOLE} \parallel 1 - i, m_2^{1-i})$.
 - (b) **Output:** Store as a secret state:

$$(\beta_i, \llbracket \tilde{r}_{1-i} \cdot \beta_i \rrbracket_i, \llbracket \tilde{\text{sk}}_{1-i} \cdot \beta_i \rrbracket_i, \tilde{r}_i, \tilde{\text{sk}}_i, \llbracket \tilde{r}_i \cdot \beta_{1-i} \rrbracket_i, \llbracket \tilde{\text{sk}}_i \cdot \beta_{1-i} \rrbracket_i, r_i) .$$

(as well as the commitment/decommitment information in the plain model).

Note that at this point, it holds that:

$$\begin{aligned} \llbracket \tilde{r}_1 \cdot \beta_0 \rrbracket_0 + \llbracket \tilde{r}_1 \cdot \beta_0 \rrbracket_1 &= \tilde{r}_1 \cdot \beta_0, & \llbracket \tilde{\text{sk}}_1 \cdot \beta_0 \rrbracket_0 + \llbracket \tilde{\text{sk}}_1 \cdot \beta_0 \rrbracket_1 &= \tilde{\text{sk}}_1 \cdot \beta_0, \\ \llbracket \tilde{r}_0 \cdot \beta_1 \rrbracket_0 + \llbracket \tilde{r}_0 \cdot \beta_1 \rrbracket_1 &= \tilde{r}_0 \cdot \beta_1, & \llbracket \tilde{\text{sk}}_0 \cdot \beta_1 \rrbracket_0 + \llbracket \tilde{\text{sk}}_0 \cdot \beta_1 \rrbracket_1 &= \tilde{\text{sk}}_0 \cdot \beta_1. \end{aligned}$$

Signing phase: The input of both parties is the joint public key pk , the individual public keys $(\text{pk}_0, \text{pk}_1)$ such that $\text{pk} = \text{pk}_0 + \text{pk}_1$ (abort if this is not the case), and the message $m \in \{0, 1\}^*$, as well as descriptions of $\text{Scalar}, \mathbb{G}$. The private input of P_0 is its share sk_0 , and the private input of P_1 is its share sk_1 . Both parties also hold their respective private state from the pre-processing phase.

1. **First round P_i for $i \in \{0, 1\}$:**

(a) Compute the fixes of the RVOLE:

$$\begin{aligned}\gamma_{r_i} &= r_i - \tilde{r}_i \\ \gamma_{\mathbf{sk}_i} &= \mathbf{sk}_i - \tilde{\mathbf{sk}}_i .\end{aligned}$$

(b) Re-define the shares:

$$\begin{aligned}\llbracket r_i \cdot \beta_{1-i} \rrbracket_i &:= \llbracket \tilde{r}_i \cdot \beta_{1-i} \rrbracket_i \\ \llbracket \mathbf{sk}_i \cdot \beta_{1-i} \rrbracket_i &:= \llbracket \tilde{\mathbf{sk}}_i \cdot \beta_{1-i} \rrbracket_i .\end{aligned}$$

(c) Compute:

$$\begin{aligned}\Gamma_{r_i} &= \llbracket r_i \cdot \beta_{1-i} \rrbracket_i \cdot G , \\ \Gamma_{\mathbf{sk}_i} &= \llbracket \mathbf{sk}_i \cdot \beta_{1-i} \rrbracket_i \cdot G .\end{aligned}$$

(d) It calls $(\text{decommit}, \text{sid} \parallel \text{com}, i)$ to F_{com} (Functionality 3.3).

(e) **Message:** It sends to P_{1-i} the message

$$m_2 = (\Gamma_{r_i}, \Gamma_{\mathbf{sk}_i}, \gamma_{r_i}, \gamma_{\mathbf{sk}_i}) .$$

and the decommitment information.

2. Second round P_i :

(a) P_i obtains the message $(\Gamma_{r_{1-i}}, \Gamma_{\mathbf{sk}_{1-i}}, \gamma_{r_{1-i}}, \gamma_{\mathbf{sk}_{1-i}})$.

(b) Given the pre-computed values $\llbracket \tilde{r}_{1-i} \cdot \beta_i \rrbracket_i$, $\llbracket \tilde{\mathbf{sk}}_{1-i} \cdot \beta_i \rrbracket_i$, β_i , define the shifted shares:

$$\begin{aligned}\llbracket r_{1-i} \cdot \beta_i \rrbracket_i &= \gamma_{r_{1-i}} \cdot \beta_i + \llbracket \tilde{r}_{1-i} \cdot \beta_i \rrbracket_i , \\ \llbracket \mathbf{sk}_{1-i} \cdot \beta_i \rrbracket_i &= \gamma_{\mathbf{sk}_{1-i}} \cdot \beta_i + \llbracket \tilde{\mathbf{sk}}_{1-i} \cdot \beta_i \rrbracket_i .\end{aligned}$$

(c) P_i receives $(\text{decommit}, \text{sid} \parallel \text{com}, 1-i, R_{1-i})$ from F_{com} .

(d) It verifies that all of the following hold:

- i. R_{1-i} is a point on the curve.
- ii. $\Gamma_{r_{1-i}}, \Gamma_{\mathbf{sk}_{1-i}}$ are points on the curve.

Otherwise, **abort**.

(e) Verify the shifted shares

$$\begin{aligned}\llbracket r_{1-i} \cdot \beta_i \rrbracket_i \cdot G &= \beta_i \cdot R_{1-i} - \Gamma_{r_{1-i}} , \\ \llbracket \mathbf{sk}_{1-i} \cdot \beta_i \rrbracket_i \cdot G &= \beta_i \cdot \mathbf{pk}_{1-i} - \Gamma_{\mathbf{sk}_{1-i}} .\end{aligned}$$

if any of the equations do not hold, **abort**.

(f) Compute:

$$R = R_0 + R_1 ,$$

and let $r^\times$ be the x -coordinate of R .

(g) Compute:

$$\begin{aligned}u_i &= r_i \cdot \beta_i + \llbracket r_i \cdot \beta_{1-i} \rrbracket_i + \llbracket r_{1-i} \cdot \beta_i \rrbracket_i , \\ v_i &= \mathbf{sk}_i \cdot \beta_i + \llbracket \mathbf{sk}_i \cdot \beta_{1-i} \rrbracket_i + \llbracket \mathbf{sk}_{1-i} \cdot \beta_i \rrbracket_i , \\ w_i &= \mathbf{H}(m) \cdot \beta_i + r^\times \cdot v_i .\end{aligned}$$

(h) **Message:** Send

$$m_3 = (u_i, w_i) .$$

3. **Compute output** P_i :

- (a) P_i receives (u_{1-i}, w_{1-i}) .
- (b) If R is invalid, or $r^\times = 0$, or $u = u_0 + u_1 = 0$, then **abort** and discard the preprocessing state.
- (c) P_i computes:
$$w = w_0 + w_1 , \quad u = u_0 + u_1 , \quad s = w/u ,$$
and **abort** if $s = 0$. Otherwise, set $\sigma = (r^\times, s)$.
- (d) Verify that $\text{Vrfy}_{\text{pk}}(m, \sigma) = 1$. If not, **abort**.
- (e) **Output:** σ .

**In all cases, after successful output or any abort,
discard the preprocessing state.**

We prove the following theorem in Appendix C:

Theorem 6.3. *Protocol 6.2 securely realizes $\mathcal{F}_{\text{Sign}}^{\text{ECDSA}}$ (Functionality 6.1) in the $(F_{\text{com}}, \mathcal{F}_{\text{RVOLE}})$ -hybrid model (Functionalities 3.3, 4.1).*

7 Evaluation and Efficiency

We evaluate the communication complexity of the different VOLE variants, as well as the communication cost of the signing protocol.

The VOLE protocol. Independently of the concrete implementation of the OT functionality, the parties must transmit the matrix $\mathbf{Q} \in \text{Scalar}^{m \times (\ell+1)}$. Recall that each element of Scalar is represented using 256 bits (ignoring serialization overhead). Table 1 summarizes the communication cost of each VOLE variant, excluding the additional communication incurred by the OTs themselves.

In our implementation, the total bandwidth is approximately doubled once the OT communication is included. Furthermore, the values reported in the table correspond to the amount of data sent by each party. Since the protocol is symmetric, each party also receives a similar amount of communication.

State size. In contrast to the communication cost of the VOLE preprocessing, the state maintained by each party after preprocessing is relatively small. Concretely, each party stores the following values:

$$(\beta_i, \llbracket \tilde{r}_{1-i} \cdot \beta_i \rrbracket_i, \llbracket \tilde{\text{sk}}_{1-i} \cdot \beta_i \rrbracket_i, \tilde{r}_i, \tilde{\text{sk}}_i, \llbracket \tilde{r}_i \cdot \beta_{1-i} \rrbracket_i, \llbracket \tilde{\text{sk}}_i \cdot \beta_{1-i} \rrbracket_i, r_i) .$$

For Secp256k1, overall, this corresponds to eight field elements, each represented using 256 bits. In addition, the parties store the decommitment information (κ bits) and the commitment to R_{1-i} (2κ bits). Hence, the total state size is

$$8 \cdot 256 + 256 + 128 = 2432 \text{ bits},$$

which is approximately 304 bytes (roughly 0.3KB).

Variant	Rounds	Receiver's input	Number of OTs	Concrete #OTs (m)	Bandwidth (excluding OTs)
I (Section 4.3)	3	End of Round 2	$\geq \log q + 3s$	436	41KB
II (Section 4.2.1)	2	End of Round 2	$\geq \log q + 2\kappa + 3s$	696	66KB
III (Section 4.4)	2	End of Round 1	$\geq \log q + 7s + 2\log(s+1) + 2s \cdot \log\left(\frac{e \cdot m}{s}\right)$	1408	135KB

Table 1: Communication complexity of the different VOLE variants. The concrete number of OTs is instantiated for Secp256k1 with $\kappa = 128$ and $s = 60$. The number of rounds refers to the VOLE protocol in the $\mathcal{F}_{m \times \text{RSOT}_{\text{Scalar}^{\ell+1}}}$ -hybrid model. The reported bandwidth excludes the communication cost of the oblivious transfers. Moreover, we consider Variant I after deriving χ using the random oracle, but sending $\mathbf{g}$ as a separate round.

Communication in the signing phase. The signing protocol consists of two rounds.

1. In the first round, each party opens its commitment by sending R_i together with the corresponding decommitment information. In addition, it sends

$$(\Gamma_{r_i}, \Gamma_{\mathbf{sk}_i}, \gamma_{r_i}, \gamma_{\mathbf{sk}_i}).$$

This corresponds to three group elements (257 bits each), two scalars (256 bits each), and decommitment information (128 bits), for a total of approximately 1400 bits.

2. In the second round, each party sends (u_i, w_i) , consisting of two scalars, for a total of 512 bits.

Overall, each party sends approximately 1900 bits during the signing phase, corresponding to roughly 0.2KB.

Acknowledgements

The author is grateful to Gil Segev, Eylon Yogev, Shahar P. Cohen and Sam Eideman for many helpful discussions.

References

- [ALSZ17] Gilad Asharov, Yehuda Lindell, Thomas Schneider, and Michael Zohner. More efficient oblivious transfer extensions. *J. Cryptol.*, 30(3):805–858, 2017. [3](#), [5](#), [9](#), [11](#)
- [BCG⁺19] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators: Silent OT extension and more. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology - CRYPTO 2019 - 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2019, Proceedings, Part III*, Lecture Notes in Computer Science, pages 489–518. Springer, 2019. [10](#)
- [BCGI18] Elette Boyle, Geoffroy Couteau, Niv Gilboa, and Yuval Ishai. Compressing vector OLE. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018, Toronto, ON, Canada, October 15-19, 2018*, pages 896–912. ACM, 2018. [10](#)
- [BDM22] Pedro Branco, Nico Döttling, and Paulo Mateus. Two-round oblivious linear evaluation from learning with errors. In *Public-Key Cryptography – PKC 2022*, volume 13177 of *Lecture Notes in Computer Science*, pages 379–408. Springer, 2022. Cryptology ePrint Archive, Report 2020/635. [10](#)
- [BEPU⁺20] Carsten Baum, Daniel Escudero, Alberto Pedrouzo-Ulloa, Peter Scholl, and Juan Ramón Troncoso-Pastoriza. Efficient protocols for oblivious linear function evaluation from ring-LWE. In *Security and Cryptography for Networks – SCN 2020*, volume 12238 of *Lecture Notes in Computer Science*, pages 130–149. Springer, 2020. [10](#)
- [BPW12] David Bernhard, Olivier Pereira, and Bogdan Warinschi. How not to prove yourself: Pitfalls of the fiat-shamir heuristic and applications to helios. In Xiaoyun Wang and Kazue Sako, editors, *Advances in Cryptology - ASIACRYPT 2012 - 18th International Conference on the Theory and Application of Cryptology and Information Security, Beijing, China, December 2-6, 2012. Proceedings*, volume 7658 of *Lecture Notes in Computer Science*, pages 626–643. Springer, 2012. [9](#)
- [Can00] Ran Canetti. Security and composition of multiparty cryptographic protocols. *J. Cryptol.*, 13(1):143–202, 2000. [10](#)
- [Can01] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd Annual Symposium on Foundations of Computer Science, FOCS 2001, 14-17 October 2001, Las Vegas, Nevada, USA*, pages 136–145. IEEE Computer Society, 2001. [10](#)
- [CGG⁺20] Ran Canetti, Rosario Gennaro, Steven Goldfeder, Nikolaos Makriyannis, and Udi Peled. UC non-interactive, proactive, threshold ECDSA with identifiable aborts. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*, pages 1769–1787. ACM, 2020. [2](#), [8](#), [9](#)

- [CMP20] Ran Canetti, Nikolaos Makriyannis, and Udi Peled. UC non-interactive, proactive, threshold ECDSA. *IACR Cryptol. ePrint Arch.*, 2020:492, 2020. 8
- [DJN⁺20] Ivan Damgård, Thomas Pelle Jakobsen, Jesper Buus Nielsen, Jakob Illeborg Pagter, and Michael Bækvang Østergård. Fast threshold ECDSA with honest majority. In Clemente Galdi and Vladimir Kolesnikov, editors, *Security and Cryptography for Networks - 12th International Conference, SCN 2020, Amalfi, Italy, September 14-16, 2020, Proceedings*, Lecture Notes in Computer Science, pages 382–400. Springer, 2020. 8
- [DKL25] DKLS.info. Information and resources related to dkls. <https://dkls.info>, 2025. Accessed: May, 2026. 2
- [DKLs18] Jack Doerner, Yashvanth Kondi, Eysa Lee, and abhi shelat. Secure two-party threshold ECDSA from ECDSA assumptions. In *2018 IEEE Symposium on Security and Privacy, SP 2018, Proceedings, 21-23 May 2018, San Francisco, California, USA*, pages 980–997. IEEE Computer Society, 2018. 2, 5, 8, 9
- [DKLs19] Jack Doerner, Yashvanth Kondi, Eysa Lee, and abhi shelat. Threshold ECDSA from ECDSA assumptions: The multiparty case. In *2019 IEEE Symposium on Security and Privacy, SP 2019, San Francisco, CA, USA, May 19-23, 2019*, pages 1051–1066. IEEE, 2019. 2, 5
- [DKLs24] Jack Doerner, Yashvanth Kondi, Eysa Lee, and abhi shelat. Threshold ECDSA in three rounds. In *IEEE Symposium on Security and Privacy, SP 2024, San Francisco, CA, USA, May 19-23, 2024*, pages 3053–3071. IEEE, 2024. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 13, 15, 16, 17, 22, 24, 25, 32, 46
- [GG18] Rosario Gennaro and Steven Goldfeder. Fast multiparty threshold ecdsa with fast trustless setup. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS '18*, page 1179–1194, New York, NY, USA, 2018. Association for Computing Machinery. 9
- [GG20] Rosario Gennaro and Steven Goldfeder. One round threshold ECDSA with identifiable abort. *IACR Cryptol. ePrint Arch.*, 2020:540, 2020. 8, 9
- [Gol04] Oded Goldreich. *The Foundations of Cryptography - Volume 2: Basic Applications*. Cambridge University Press, 2004. 10
- [GS22] Jens Groth and Victor Shoup. On the security of ECDSA with additive key derivation and presignatures. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology - EUROCRYPT 2022 - 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Trondheim, Norway, May 30 - June 3, 2022, Proceedings, Part I*, Lecture Notes in Computer Science, pages 365–396. Springer, 2022. 3, 8
- [HLNR18] Iftach Haitner, Yehuda Lindell, Ariel Nof, and Samuel Ranellucci. Fast secure multiparty ECDSA with practical distributed key generation and applications to cryptocurrency custody. *Cryptology ePrint Archive*, Paper 2018/987, 2018. 2, 9

- [HLPT20] Thomas Haines, Sarah Jamie Lewis, Olivier Pereira, and Vanessa Teague. How not to prove your election outcome. In *2020 IEEE Symposium on Security and Privacy, SP 2020, San Francisco, CA, USA, May 18-21, 2020*, pages 644–660. IEEE, 2020. 9
- [ILL89] Russell Impagliazzo, Leonid A. Levin, and Michael Luby. Pseudo-random generation from one-way functions (extended abstracts). In David S. Johnson, editor, *Proceedings of the 21st Annual ACM Symposium on Theory of Computing, May 14-17, 1989, Seattle, Washington, USA*, pages 12–24. ACM, 1989. 4, 18
- [IN96] Russell Impagliazzo and Moni Naor. Efficient cryptographic schemes provably as secure as subset sum. *J. Cryptol.*, 9(4):199–216, 1996. 4
- [KOS15] Marcel Keller, Emmanuela Orsini, and Peter Scholl. Actively secure OT extension with optimal overhead. *IACR Cryptol. ePrint Arch.*, page 546, 2015. 5
- [KU25] Jonathan Katz and Antoine Urban. Honest-majority threshold ECDSA with batch generation of key-independent presignatures. *IACR Commun. Cryptol.*, 2(1):8, 2025. 2, 9
- [Lin17] Yehuda Lindell. Fast secure two-party ECDSA signing. In Jonathan Katz and Hovav Shacham, editors, *Advances in Cryptology - CRYPTO 2017 - 37th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 20-24, 2017, Proceedings, Part II*, volume 10402 of *Lecture Notes in Computer Science*, pages 613–644. Springer, 2017. 2, 9
- [Lin24] Yehuda Lindell. Simple three-round multiparty schnorr signing with full simulatability. *IACR Commun. Cryptol.*, 1(1):25, 2024. 8, 11, 24
- [MR19] Daniel Masny and Peter Rindal. Endemic oblivious transfer. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS 2019, London, UK, November 11-15, 2019*, pages 309–326. ACM, 2019. 3, 8, 16
- [OSY21] Claudio Orlandi, Peter Scholl, and Sophia Yakoubov. The rise of paillier: Homomorphic secret sharing and public-key silent OT. In *Advances in Cryptology - EURO-CRYPTO 2021*, volume 12696 of *Lecture Notes in Computer Science*, pages 678–708. Springer, 2021. 10
- [Roy22] Lawrence Roy. Softspokenot: Quieter OT extension from small-field silent VOLE in the minicrypt model. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology - CRYPTO 2022 - 42nd Annual International Cryptology Conference, CRYPTO 2022, Santa Barbara, CA, USA, August 15-18, 2022, Proceedings, Part I*, volume 13507 of *Lecture Notes in Computer Science*, pages 657–687. Springer, 2022. 5
- [vdP23] Joop van de Pol. Don’t overextend your oblivious transfer. The Trail of Bits Blog, 2023. <https://blog.trailofbits.com/2023/09/20/dont-overextend-your-oblivious-transfer/>, accessed: May 2026. 5
- [Wui12] Pieter Wuille. Hierarchical deterministic wallets, 2012. <https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki>, accessed: May 2026. 8

A Experiment on Small Parameters

Our analysis addresses that one cannot apply the leftover-hash lemma directly in the original protocol of [DKLs24]. However, this does not rule out the possibility that the parameter choice given in [DKLs24] is correct, but requires some new proof.

In this section, we investigate whether the parameter choice in [DKLs24] is correct. We show that for small parameters, it does not achieve the intended security level. That is, we show that for a random $\mathbf{g} \in \text{Scalar}^m$, there exists some constraint $\mathbf{c}$, which defines a distribution $W_{\mathbf{c}}$ such that $\text{SD}(\mathbf{g}(W_{\mathbf{c}}), U_{\mathbb{Z}_q}) > 2^{-s}$.

Towards that end, we implemented a small script in Python that runs over all possibilities. Of course, such code takes a long time to run (quantifying over all possible $\mathbf{g}$ s), and so we can run it over small parameters only. We consider:

1. $\mathbb{Z}_q = \{0, \dots, 6\}$, i.e., $q = 7$. This implies that $\log q = 3$.
2. We took $s = 2$.
3. While in our security game, the adversary is allowed to output s constraints, we run with fixing just a single constraint.
4. Quantifying over all $\mathbf{g}$, we computed whether there exists a (single) constraint $\mathbf{c}$ for which $\text{SD}(\mathbf{g}(W_{\mathbf{c}}), U_{\mathbb{Z}_q}) > 2^{-s}$, where $U_{\mathbb{Z}_q}$ is the uniform distribution over $\mathbb{Z}_q$.
5. We took the parameters suggested by [DKLs24]. That is, $m = \log q + 2s = 3 + 2 \cdot 2 = 7$.
6. For each $\mathbf{g}$, we say it is “bad” if there exists $\mathbf{c}$ for which $\text{SD}(\mathbf{g}(W_{\mathbf{c}}), U_{\mathbb{Z}_q}) > 2^{-s}$, and “good” otherwise.
7. For that set of parameters, the number of bad $\mathbf{g}$ is 220087, out of 823543 possible functions. That is, more than 26% of the possible functions have a bad set of constraints that result in a larger statistical distance than required.
8. We remark that this is when the adversary is allowed to provide a single constraint; In the analysis, it is allowed to provide up to s constraints.

The above shows that the bounds provided in [DKLs24] are too liberal.

B Full Proofs for the VOLE Protocols

B.1 Security Analysis for Variant I

Theorem B.1. *Assuming that:*

1. $\log q \geq \kappa + 2 \log m$,
2. $m \geq \log q + 3 \cdot s$,

then the “more-rounds” protocol described in Section 4.3 (Variant I) securely realizes the $\mathcal{F}_{\text{RVOLE}}$ functionality (Functionality 4.1) in the $\mathcal{F}_{m \times \text{RSOT}_{\text{Scalar}^{\ell+1}}}$ -hybrid model.

We separate between the case of a corrupted sender and a corrupted receiver, as well as when both parties are honest.

B.1.1 Correctness

We start with the case where no party is corrupted, and show that the output of the honest parties in the real execution is like in the ideal world. In the ideal world, when no party is corrupted, the functionality samples $b = \beta$ at random, samples $\mathbf{t} = (t_1, \dots, t_\ell)$ at random, samples $\alpha = (\alpha_1, \dots, \alpha_\ell)$ at random, and sets for every $i \in [\ell]$: $r_i = \alpha_i \cdot b - t_i$.

In the real world, b is statistically-close to a random element in Scalar , since $b = \langle \mathbf{g}, \mathbf{w} \rangle$, and $\mathbf{w}$ is uniform. See also Section 5. Moreover, observe that $\mathcal{F}_{\mathbf{m} \times \text{RSOT}}_{\text{Scalar}^{\ell+1}}$ guarantees that:

$$\mathbf{S}_{j,i} = (1 - w_j) \cdot \mathbf{T}_{j,i}^0 + w_j \cdot \mathbf{T}_{j,i}^1 = \mathbf{T}_{j,i}^0 + w_j \cdot (\mathbf{T}_{j,i}^1 - \mathbf{T}_{j,i}^0).$$

Therefore,

$$\begin{aligned} \mathbf{R}_{j,i} &= \mathbf{S}_{j,i} + w_j \cdot \mathbf{Q}_{j,i} \\ &= \mathbf{S}_{j,i} + w_j \cdot (\mathbf{T}_{j,i}^0 - \mathbf{T}_{j,i}^1 + \mathbf{A}_{j,i}) \\ &= \mathbf{T}_{j,i}^0 + w_j \cdot (\mathbf{T}_{j,i}^1 - \mathbf{T}_{j,i}^0) + w_j \cdot (\mathbf{T}_{j,i}^0 - \mathbf{T}_{j,i}^1 + \mathbf{A}_{j,i}) \\ &= \mathbf{T}_{j,i} + w_j \cdot \mathbf{A}_{j,i}. \end{aligned} \tag{3}$$

This implies that Eq. (1) indeed holds. The output shares of the parties satisfy for every $i \in [\ell]$:

$$\begin{aligned} t_i + r_i &= - \sum_{j=1}^m g_j \cdot t_{j,i} + \sum_{j=1}^m g_j \cdot r_{j,i} \\ &= \sum_{j=1}^m g_j \cdot (r_{j,i} - t_{j,i}) \\ &= \sum_{j=1}^m g_j \cdot (w_j \cdot \alpha_i) \\ &= \alpha_i \cdot \langle \mathbf{g}, \mathbf{w} \rangle \\ &= \alpha_i \cdot b. \end{aligned}$$

Moreover, each t_i is uniform in Scalar because $\mathbf{T}^0$ is random, as follows from the definition of $\mathcal{F}_{\mathbf{m} \times \text{RSOT}}_{\text{Scalar}^{\ell+1}}$ (see Functionality 3.4).

B.1.2 Malicious Sender

Claim B.2. *The protocol described in Section 4.3 securely realizes the $\mathcal{F}_{\text{RVOLE}}$ functionality (Functionality 4.1) in the $\mathcal{F}_{\text{RSOT}}$ -hybrid model in the presence of a malicious **Sender**.*

Proof. We now consider the simulation of VOLE:

Simulator B.3: Simulating Malicious Sender

1. On receiving $(\text{multiply}, \text{sid})$ from $\mathcal{F}_{\text{RVOLE}}$, simulate $\mathcal{F}_{\mathbf{m} \times \text{RSOT}}_{\text{Scalar}^{\ell+1}}$ (Functionality 3.4) to the adversary:
 - (a) Choose $\mathbf{T}^0, \mathbf{T}^1$ uniformly at random in $\text{Scalar}^{\mathbf{m} \times (\ell+1)}$.
 - (b) Send $(\text{senderInput}, \text{sid} \parallel \text{RSOT}, \mathbf{T}^0, \mathbf{T}^1)$ to $\mathcal{A}$ as sent from $\mathcal{F}_{\mathbf{m} \times \text{RSOT}}_{\text{Scalar}^{\ell+1}}$.

2. Receive from $\mathcal{A}$ its message to the **Receiver** – $\mathbf{Q}$.
3. Extract $\mathbf{A} = \mathbf{Q} - \mathbf{T}^0 + \mathbf{T}^1$, and denote:

$$\mathbf{A} = \begin{pmatrix} \alpha_{1,1} & \dots & \alpha_{1,\ell} & \hat{\alpha}_1 \\ \alpha_{2,1} & \dots & \alpha_{2,\ell} & \hat{\alpha}_2 \\ & \dots & & \dots \\ \alpha_{\mathbf{m},1} & \dots & \alpha_{\mathbf{m},\ell} & \hat{\alpha}_{\mathbf{m}} \end{pmatrix}.$$

Note that if $\mathcal{A}$ behaves honestly, then all the rows are the same and define some $\boldsymbol{\alpha} = (\alpha_1, \dots, \alpha_\ell, \hat{\alpha})$.

4. Moreover, set $\mathbf{T} = \mathbf{T}^0$.
5. **Simulating round 3 of the Receiver:** Pick $(\chi_1, \dots, \chi_\ell)$ uniformly at random in Scalar^ℓ and send it to the adversary. Set $\boldsymbol{\chi} = (\chi_1, \dots, \chi_\ell, 1)$.
6. **Round 4:** $\mathcal{A}$ sends $\mathbf{u}, v$ as its message in round 4.
7. **Verification step:** The simulator performs a verification step without explicitly defining $\mathbf{w}$:
 - (a) Define the matrix $\mathbf{u}^*$ that the sender was supposed to compute. That is, let

$$\mathbf{u}^* = \begin{pmatrix} u_1^* \\ \dots \\ u_{\mathbf{m}}^* \end{pmatrix} = \mathbf{T} \cdot \boldsymbol{\chi} = \begin{pmatrix} \chi_1 \cdot t_{1,1} + \dots + \chi_\ell \cdot t_{1,\ell} + \hat{t}_1 \\ \dots \\ \chi_1 \cdot t_{\mathbf{m},1} + \dots + \chi_\ell \cdot t_{\mathbf{m},\ell} + \hat{t}_{\mathbf{m}} \end{pmatrix}.$$

- (b) Compute the vector of offsets from the expectation. That is:

$$\begin{aligned} \boldsymbol{\Delta} &= \begin{pmatrix} \Delta_1 \\ \dots \\ \Delta_{\mathbf{m}} \end{pmatrix} = \begin{pmatrix} \langle \mathbf{A}_1, \boldsymbol{\chi} \rangle - v \\ \dots \\ \langle \mathbf{A}_{\mathbf{m}}, \boldsymbol{\chi} \rangle - v \end{pmatrix} \\ &= \begin{pmatrix} \chi_1 \cdot \alpha_{1,1} + \dots + \chi_\ell \cdot \alpha_{1,\ell} + \hat{\alpha}_1 - v \\ \dots \\ \chi_1 \cdot \alpha_{\mathbf{m},1} + \dots + \chi_\ell \cdot \alpha_{\mathbf{m},\ell} + \hat{\alpha}_{\mathbf{m}} - v \end{pmatrix}. \end{aligned}$$

Note that if $\mathcal{A}$ behaves honestly, then this vector is the all-zero vector.

- (c) Looking ahead, the proof would show that the verification step in the real is equivalent to the following. For every $j \in [\mathbf{m}]$ it must hold that:

$$u_j - u_j^* = w_j \cdot \Delta_j.$$

While the simulator does not explicitly define $\mathbf{w}$, we find now what constraints $\mathbf{u}, v$ impose on $\mathbf{w}$, and for which $\mathbf{w}$ the verification works. Initialize a set $\text{constraints} = \emptyset$. For every $j \in [\mathbf{m}]$:

- i. If $\Delta_j = 0$ and $u_j^* = u_j$ then there is no constraint imposed on w_j .
- ii. If $\Delta_j = 0$ and $u_j^* \neq u_j$ then it always holds that $u_j - u_j^* \neq 0$ but $w_j \cdot \Delta_j = 0$, and thus the verification of $u_j - u_j^* = w_j \cdot \Delta_j$ fails. The simulator aborts.
- iii. If $\Delta_j \neq 0$ then we can re-arrange $w_j = \Delta_j^{-1} \cdot (u_j - u_j^*)$. Then:
 - A. If $u_j^* = u_j$ then add $(j, 0)$ to constraints .
 - B. If $u_j^* \neq u_j$ then verify that $\Delta_j^{-1} \cdot (u_j - u_j^*) = 1$, otherwise, the simulator aborts, as $w_j \in \{0, 1\}$. Then, add $(j, 1)$ to constraints .

- (d) If $|\text{constraints}| \geq s$, then **abort**.
 - (e) Toss a coin that is head with probability $2^{-|\text{constraints}|}$. If head - continue. Otherwise, **abort**.
8. **Extracting the input:**
- (a) Initialize a set $S = \emptyset$. Given the matrix $\mathbf{A}$ that was extracted in Step 3 of the simulation, we extract the input $(\alpha_1, \dots, \alpha_\ell, \hat{\alpha})$. Towards that end, for every $j \in [m]$ add $\mathbf{A}_j = (\alpha_{j,1}, \dots, \alpha_{j,\ell}, \hat{\alpha}_j)$ to S if

$$\chi_1 \cdot \alpha_{j,1} + \dots + \chi_\ell \cdot \alpha_{j,\ell} + \hat{\alpha}_j = v .$$
 - (b) If $|S| \neq 1$ - **abort**.
 - (c) Let $(\alpha_1, \dots, \alpha_\ell, \hat{\alpha})$ be the unique entry in S .
9. **Sampling $\mathbf{g}$:**
 Sample $\mathbf{g}$ uniformly at random in Scalar^m as the honest receiver in the real.
10. **Computing the outputs $(t_1, \dots, t_\ell)$.**
- (a) Define

$$J = \{j \in [m] \mid (j, 1) \in \text{constraints}\} .$$
 and let

$$d_i = - \sum_{j \in J} g_j \cdot (\alpha_{j,i} - \alpha_i) .$$
 - (b) Compute

$$t_i = d_i - \sum_{j=1}^m g_j \cdot t_{j,i} .$$
11. Send to the adversary the message $\mathbf{g} = (g_1, \dots, g_m)$.
12. Send to $\mathcal{F}_{\text{RVOLE}}$ (Functionality 4.1) the instruction $(\text{senderInput}, \text{sid}, \alpha, \mathbf{t})$.
-

We now prove that the real and ideal are identically distributed via a sequence of hybrids:

Hyb_{S1}: This is the real execution.

Hyb_{S2}: We replace the honest **Receiver** and $\mathcal{F}_{\text{RST}}$ with one machine that runs the receiver, and simulates the $\mathcal{F}_{\text{RST}}$ functionality.

Hyb_{S3}: In this execution, instead of the receiver aborting as in the real while verifying that for all $j \in [m]$ it holds that $u_j = u'_j$ as in Step 3d in the protocol, it defines $\mathbf{u}^*$ and Δ as in Step 7 in the simulation, and performs the following verification that for every $j \in [m]$, and aborts otherwise:

$$u_j - u_j^* = w_j \cdot \Delta_j .$$

Hyb_{S4}: Instead of verifying that for all $j \in [m]$ it holds that $u_j - u_j^* = w_j \cdot \Delta_j$, the simulator generates **constraints** as described in the simulation. It then sample $\mathbf{w}$ uniformly at random, and checks that it is consistent with **constraints**. If not, it aborts.

Hyb_{S5}: Here we add also step 8 in the simulation. That is, given the matrices $\mathbf{T}^0, \mathbf{T}^1$ that $\mathcal{A}$ received from $\mathcal{F}_{m \times \text{RSOT}_{\text{Scalar}^{\ell+1}}}$, and the matrix $\mathbf{Q}$ that it sent to the honest **Receiver**, extract $\mathbf{A} = \mathbf{Q} - \mathbf{T}^0 + \mathbf{T}^1$. Then, we extract $(\alpha_1, \dots, \alpha_\ell, \hat{\alpha})$ as described in the simulation. That is, for every $j \in [m]$, check whether $\langle \mathbf{A}_j, \chi \rangle = v$, and if so, add $\mathbf{A}_j$ to S . Abort if $|S| \neq 1$.

Hyb_{S6}: In this hybrid, we change the output of the honest receiver. In the previous hybrid, this is being computed as $b = \langle \mathbf{g}, \mathbf{w} \rangle$, and then for every $i \in [\ell]$:

$$r_i = \sum_{j=1}^m g_j \cdot r_{j,i}$$

In this hybrid, we change it to be computed closer to the ideal model. Namely, let

$$d_i = - \sum_{j=1}^m g_j \cdot w_j \cdot (\alpha_{j,i} - \alpha_i) .$$

Compute:

$$t_i = d_i - \sum_{j=1}^m g_j \cdot t_{j,i} .$$

The simulator then computes $b = \langle \mathbf{g}, \mathbf{w} \rangle$ as before, and computes

$$r_i = \alpha_i \cdot b - t_i .$$

Hyb_{S7}: In this hybrid, we change the definition of d_i . Instead of defining:

$$d_i = - \sum_{j=1}^m g_j \cdot w_j \cdot (\alpha_{j,i} - \alpha_i)$$

as in the previous hybrid, we define $J = \{j \in [m] \mid (j, 1) \in \text{constraints}\}$ and then:

$$d_i = - \sum_{j \in J} g_j \cdot (\alpha_{j,i} - \alpha_i) .$$

Note that now, $t_1, \dots, t_\ell$ are not computed using $\mathbf{w}$.

Hyb_{S8}: In this hybrid, we change the condition to abort. Instead of sampling $\mathbf{w}$ uniformly at random, and then verifying whether $\mathbf{w}$ is consistent with **constraints**, we proceed as follows. (1) We sample a coin coin with probability $1 - 2^{-|\text{constraints}|}$ to be 1. If $\text{coin} = 1$, then we abort; otherwise: (2) We sample $\mathbf{w}^*$ uniformly at random under the constraint that it agrees with **constraints**. Namely, we simply fix all locations in which $(j, \sigma) \in \text{constraints}$ as w_j^* , for some $\sigma \in \{0, 1\}$, and sample all the other locations uniformly at random.

Note that b is defined according to $\mathbf{w}^*$ as $b = \langle \mathbf{g}, \mathbf{w}^* \rangle$ and is used to determine the output of the honest receiver $(r_1, \dots, r_\ell)$.

Hyb_{S9}: In this hybrid, we add one more condition to abort. If $|\text{constraints}| \geq s$, then the simulator always aborts.

Hyb_{S10}: In this hybrid, we change the output of the honest receiver. Instead of the simulator determining the output by computing $b = \langle \mathbf{g}, \mathbf{w} \rangle$ and then $r_i = \alpha_i \cdot b - t_i$, it lets the trusted party determine it. The simulator sends $(\alpha_1, \dots, \alpha_\ell)$ and $(t_1, \dots, t_\ell)$ to the trusted party. The trusted party samples b uniformly in **Scalar**. The trusted party defines $r_i = \alpha_i \cdot b - t_i$.

Note that the simulator now never defines $\mathbf{w}$, nor b . This is the ideal execution.

We now show that every pair of consecutive hybrids is statistically-close, or identically-distributed.

Claim B.4. **Hyb_{S1}** and **Hyb_{S2}** are identically distributed.

Proof. The difference between the two hybrids is purely syntactical. $\square$

Claim B.5. **Hyb_{S2}** and **Hyb_{S3}** are identically distributed.

Proof. First, recall that the honest receiver defines

$$\begin{aligned} \mathbf{R}_{j,i} &= \mathbf{S}_{j,i} + w_j \cdot \mathbf{Q}_{j,i} \\ &= \mathbf{T}_{j,i}^0 + w_j \cdot (\mathbf{T}_{j,i}^1 - \mathbf{T}_{j,i}^0) + w_j \cdot \mathbf{Q}_{j,i} \\ &= \mathbf{T}_{j,i} + w_j \cdot \mathbf{A}_{j,i} , \end{aligned}$$

where the first step holds since $\mathbf{S}_{j,i} = \mathbf{T}_{j,i}^0 + w_j \cdot (\mathbf{T}_{j,i}^1 - \mathbf{T}_{j,i}^0)$, last step holds since the simulator defines $\mathbf{A} = \mathbf{Q} - \mathbf{T}^0 + \mathbf{T}^1$, and $\mathbf{T} = \mathbf{T}^0$.

In **Hyb_{S2}**, the receiver computes for every $j \in [m]$:

$$u'_j = \langle \mathbf{R}_j, \boldsymbol{\chi} \rangle - w_j \cdot v .$$

and let $\mathbf{u}' = (u'_1, \dots, u'_m)$. It then verifies that for every $j \in [m]$ it holds that $u_j = u'_j$. That is, it computes:

$$\begin{aligned} u'_j &= \langle \boldsymbol{\chi}, \mathbf{R}_j \rangle - w_j \cdot v \\ &= \langle \boldsymbol{\chi}, \mathbf{T}_j + w_j \cdot \mathbf{A}_j \rangle - w_j \cdot v \\ &= \langle \boldsymbol{\chi}, \mathbf{T}_j \rangle + w_j \cdot (\langle \boldsymbol{\chi}, \mathbf{A}_j \rangle - v) \\ &= \langle \boldsymbol{\chi}, \mathbf{T}_j \rangle + w_j \cdot \Delta_j \\ &= u_j^* + w_j \cdot \Delta_j \end{aligned}$$

and verifies that $u_j = u'_j$, i.e.,

$$u_j = u_j^* + w_j \cdot \Delta_j$$

In **Hyb_{S3}** the simulator performs the following verification:

$$u_j - u_j^* = w_j \cdot \Delta_j ,$$

which is exactly the same verification. $\square$

Claim B.6. **Hyb_{S3}** and **Hyb_{S4}** are identical.

Proof. In **Hyb_{S3}**, the simulator picks $\mathbf{w}$ uniformly at random, and then verify that for every j , $u_j - u_j^* = w_j \cdot \Delta_j$. In **Hyb_{S4}** the simulator instead adds constraints to **constraints**, then it picks $\mathbf{w}$ uniformly at random, and verifies that $\mathbf{w}$ is consistent with **constraints**.

By inspection, **Hyb_{S3}** aborts iff **Hyb_{S4}** aborts. Specifically:

1. If $\Delta_j = 0$ and $u_j^* = u_j$, then there is no constraint added in **Hyb_{S4}**. In that case, no matter what w_j is, it always holds that $0 = u_j - u_j^* = w_j \cdot \Delta_j = 0$, and **Hyb_{S3}** does not abort due to index j .
2. If $\Delta_j = 0$ and $u_j \neq u_j^*$, then both hybrids always abort. In **Hyb_{S4}**, this is explicit. In **Hyb_{S3}**, then $0 \neq u_j - u_j^* = w_j \cdot \Delta_j = 0$ regardless of whether $w_j = 0$ or $w_j = 1$. The simulator aborts.
3. If $\Delta_j \neq 0$ and $u_j^* = u_j$, then **Hyb_{S4}** aborts iff $w_j = 1$. In **Hyb_{S3}**, if indeed $w_j = 1$ then $0 = u_j - u_j^* = 1 \cdot \Delta_j \neq 0$, and thus the simulator aborts.
4. If $\Delta_j \neq 0$, and $u_j^* \neq u_j$, but $\Delta_j^{-1}(u_j - u_j^*) \neq 1$ then both simulators abort. In **Hyb_{S4}** this is explicit. In **Hyb_{S3}**:
 - (a) If $w_j = 0$, then $u_j - u_j^* = w_j \cdot \Delta_j$ fails since $0 \neq u_j - u_j^* = 0$.
 - (b) If $w_j = 1$, then $u_j - u_j^* = w_j \cdot \Delta_j$ fails since $u_j - u_j^* \neq 1 \cdot \Delta_j$.
5. If $\Delta_j \neq 0$, and $u_j^* \neq u_j$, and $\Delta_j^{-1}(u_j - u_j^*) = 1$ then:
 - (a) If $w_j = 0$, then both simulators abort. In **Hyb_{S4}**, the simulator adds the constraint $w_j = 1$ and fails in the verification of the constraints. In **Hyb_{S3}**, the simulator verifies $u_j - u_j^* = w_j \cdot \Delta_j = 0$, but it holds that $u_j - u_j^* \neq 0$.
 - (b) If $w_j = 1$, then **Hyb_{S4}** does not abort due to j ; the simulator adds $w_j = 1$ as a constraint, and when choosing w_j at random it indeeds holds that $w_j = 1$; In **Hyb_{S3}**, it holds that $u_j - u_j^* = w_j \cdot \Delta_j = \Delta_j$ and the simulator does not abort due to this verification.

□

Claim B.7. **Hyb_{S4}** and **Hyb_{S5}** are $(2^{-m} + 2^{-\kappa})$ -statistically close.

Proof. In **Hyb_{S5}**, there is an additional check that does not exist in **Hyb_{S4}**. In particular, we abort if $|S| \neq 1$.

If $|S| = 0$, then this implies that for every $j \in [m]$, we have that $\Delta_j \neq 0$. There are three cases:

1. Case 1: if $u_j = u_j^*$ then this adds the constraint $w_j = 0$ to constraints.
2. Case 2: if $u_j \neq u_j^*$ and $\Delta_j^{-1} \cdot (u_j - u_j^*) \neq 1$ then both hybrids always abort.
3. Case 3: if $u_j \neq u_j^*$ and $\Delta_j^{-1} \cdot (u_j - u_j^*) = 1$ then the simulator adds $w_j = 1$ to constraints.

Hyb_{S5} always abort if $|S| \neq 1$. There are two cases to consider:

1. If $|S| = 0$, then for every $j \in [m]$, there exists a constraint $(j, \sigma) \in \text{constraints}$. Thus, constraints contains m constraints, one to each index $j \in [m]$. **Hyb_{S4}** then sample $\mathbf{w}$ and verifies that it is consistent with constraints. Since there are m constraints, it does not abort with probability 2^{-m} .
2. If $|S| > 1$, then there must exist two rows, $\mathbf{A}_i, \mathbf{A}_j$, such that:
 - (a) $\langle \mathbf{A}_i, \chi \rangle = \langle \mathbf{A}_j, \chi \rangle = v$,
 - (b) $\mathbf{A}_i \neq \mathbf{A}_j$.

Thus, for a pair $(\mathbf{A}, \chi)$ we say that the pair is **bad** if there exists $i, j \in [m]$ such that $\mathbf{A}_i \neq \mathbf{A}_j$, but $\langle \mathbf{A}_i, \chi \rangle = \langle \mathbf{A}_j, \chi \rangle$. We show the following claim:

Claim B.8. Assuming that $\log q \geq \kappa + 2 \log m$, for every $\mathbf{A}$:

$$\Pr_{\chi}[(\mathbf{A}, \chi) \text{ is bad}] \leq 2^{-\kappa}.$$

Proof. For every fixed $\mathbf{A}_i \neq \mathbf{A}_j$ there must hold an index k such that $\alpha_{i,k} \neq \alpha_{j,k}$, and thus $\alpha_{i,k} - \alpha_{j,k}$ has an inverse. Thus, we can write:

$$\Pr_{\chi} [\langle \mathbf{A}_i, \chi \rangle = \langle \mathbf{A}_j, \chi \rangle] = \Pr_{\chi} \left[\chi_k = (\alpha_{i,k} - \alpha_{j,k})^{-1} \cdot \left(\sum_{s \neq k} \chi_s \cdot (\alpha_{j,s} - \alpha_{i,s}) \right) \right] = \frac{1}{|\text{Scalar}|}.$$

By a union bound over all pairs of rows of $\mathbf{A}$, we obtain that:

$$\Pr [\exists i, j \in [m] \text{ s.t. } \mathbf{A}_i \neq \mathbf{A}_j \wedge \langle \mathbf{A}_i, \chi \rangle = \langle \mathbf{A}_j, \chi \rangle] \leq m^2 \cdot |\text{Scalar}|^{-1} \leq 2^{-\kappa},$$

where the latter holds as long as $|\text{Scalar}| = 2^{\log q}$, and $\log q \geq \kappa + 2 \log m$. $\square$

We therefore obtain that the difference between the two hybrids is bounded by $2^{-m} + 2^{-\kappa}$. $\square$

Claim B.9. Hyb_{S5} and Hyb_{S6} are identically distributed.

Proof. In Hyb_{S5} the output of the honest receiver is computed as (for every $i \in [\ell]$):

$$r_i = \sum_{j=1}^m g_j \cdot r_{j,i},$$

where (1) $b = \langle \mathbf{g}, \mathbf{w} \rangle$ where the honest receiver chooses $\mathbf{w}$ uniformly at random in $\{0, 1\}^m$, and, conditioned on no abort, it holds that $\mathbf{w}$ is consistent with **constraints**; (2) we saw that $r_{j,i} - t_{j,i} = w_j \cdot \alpha_{j,i}$.

In Hyb_{S6} , the simulator computes the supposed output of the corrupted sender as:

$$\begin{aligned} t_i &= d_i - \sum_{j=1}^m g_j \cdot t_{j,i} \\ &= - \sum_{j=1}^m g_j \cdot w_j \cdot (\alpha_{j,i} - \alpha_i) - \sum_{j=1}^m g_j \cdot (r_{j,i} - \alpha_{j,i} \cdot w_j) \\ &= \sum_{j=1}^m g_j \cdot w_j \cdot \alpha_i - \sum_{j=1}^m g_j \cdot r_{j,i} \\ &= \alpha_i \cdot b - \sum_{j=1}^m g_j \cdot r_{j,i} \end{aligned}$$

It then defines the output of the honest party as

$$\begin{aligned} r_i &= \alpha_i \cdot b - t_i \\ &= \alpha_i \cdot b - \alpha_i \cdot b + \sum_{j=1}^m g_j \cdot r_{j,i} \\ &= \sum_{j=1}^m g_j \cdot r_{j,i} \end{aligned}$$

which is exactly the way the output of the honest receiver is defined in Hyb_{S5} . $\square$

Claim B.10. $\mathbf{Hyb}_{S6}$ and $\mathbf{Hyb}_{S7}$ are identically distributed.

Proof. We claim that for every $\mathbf{w}$ that is consistent with **constraints** it holds that:

$$d_i = - \sum_{j \in J} g_j \cdot (\alpha_{j,i} - \alpha_i) = - \sum_{j=1}^m g_j \cdot w_j \cdot (\alpha_{j,i} - \alpha_i)$$

where recall that $J = \{j \in [m] \mid (j, 1) \in \mathbf{constraints}\}$. Assuming both hybrids do not abort, recall that $u_j - u_j^* = w_j \cdot \Delta_j$. We have:

1. For every $j \in [m]$ such that $\Delta_j = 0$, i.e., $\langle \mathbf{A}_j, \chi \rangle = v$, it must hold that $\alpha_{j,i} = \alpha_i$. This holds from the fact that $|S| = 1$, and recall that both hybrids abort if $|S| \neq 1$. As such, $(\alpha_{j,i} - \alpha_i) = 0$, and we can ignore j in the summation.
2. For every $j \in [m]$ such that $\Delta_j \neq 0$, there are two cases, conditioned on no abort:
 - (a) If $u_j^* = u_j$ then it must hold that $w_j = 0$, and we can ignore j in the summation.
 - (b) If $u_j^* \neq u_j$ and no abort, then $(j, 1) \in \mathbf{constraints}$. Therefore, for every $\mathbf{w}$ in the support, for which the hybrids do not abort:

$$g_j \cdot (\alpha_{j,i} - \alpha_i) = g_j \cdot w_j (\alpha_{j,i} - \alpha_i) .$$

□

Claim B.11. $\mathbf{Hyb}_7$ and $\mathbf{Hyb}_8$ are identically distributed.

Proof. The difference is just syntactic.

In $\mathbf{Hyb}_{S7}$ we sample $\mathbf{w}$ uniformly at random from $\{0, 1\}^m$. Then:

1. We check whether $\mathbf{w}$ is consistent with **constraints**. For a random $\mathbf{w}$, this occurs with probability $2^{-|\mathbf{constraints}|}$.
2. If it is not consistent (with probability $1 - 2^{-|\mathbf{constraints}|}$) we abort.
3. Otherwise, we proceed. Note that this implies that for every constraint $(w_j = \rho) \in \mathbf{constraints}$ it must hold that $w_j = \rho$.

In $\mathbf{Hyb}_{S8}$ we instead choose whether to abort with probability $1 - 2^{-|\mathbf{constraints}|}$. Then, if not aborted, we proceed with picking $\mathbf{w}^*$ such that it must agree with **constraints**. This means fixing all the indices that there are constraints, and choosing the rest uniformly at random. □

Claim B.12. $\mathbf{Hyb}_{S8}$ and $\mathbf{Hyb}_{S9}$ are 2^{-s} statistically-close.

Proof. The difference between the two hybrids is as follows. If $|\mathbf{constraints}| \geq s$, then $\mathbf{Hyb}_{S9}$ always aborts. On the other hand, $\mathbf{Hyb}_{S8}$ samples $\mathbf{w}^*$ and verifies whether $\mathbf{w}^*$ is consistent with **constraints**, and if not, it aborts. If $|\mathbf{constraints}| \geq s$ then the probability that $\mathbf{Hyb}_{S8}$ does not abort is smaller than 2^{-s} . Thus, the difference between the two hybrids is at most 2^{-s} . □

Claim B.13. $\mathbf{Hyb}_{S9}$ and $\mathbf{Hyb}_{S10}$ are 2^{-s} -statistically close.

Proof. The only difference between the two hybrids is in the way b is defined.

- In **Hyb_{S9}**, the adversary first outputs its messages, which define a set of constraints **constraints** on $\mathbf{w}$. The choice whether to abort is determined by the size of **constraints**. The simulator picks $\mathbf{g}$ uniformly at random, and $\mathbf{w}$ uniformly at random (under the constraint that it is consistent with **constraints**), and sets $b = \langle \mathbf{g}, \mathbf{w} \rangle$. The value b is then used to determine $r_1, \dots, r_\ell$ where each $r_i = \alpha_i \cdot b - t_i$.
- In **Hyb_{S10}**, the adversary also output a set of **constraints**. The choice whether to abort is determined by the size of **constraints**. Conditioning on no abort, the trusted party picks b uniformly at random in Scalar , and also sets $r_i = \alpha_i \cdot b - t_i$ for every $i \in [\ell]$.

The two hybrids are 2^{-s} -statistically close by Claim 5.3. $\square$

From combining all hybrids, we obtain that the distinguishing probability between the real and ideal is bounded by:

$$(2^{-m} + 2^{-\kappa}) + 2^{-s} + 2^{-s}.$$

$\square$

B.1.3 Malicious Receiver

Claim B.14. *The protocol described in Section 4.3 securely realizes the $\mathcal{F}_{\text{RVOLE}}$ functionality (Functionality 4.1) in the $\mathcal{F}_{\text{RSOT}}$ -hybrid model in the presence of a malicious **Receiver**.*

Proof. We start with providing a simulator for the case of a corrupted **Receiver**.

Simulator B.15: Simulating malicious Receiver:

Multiplication: On receiving $(\text{multiply}, \text{sid})$ from $\mathcal{F}_{\text{RVOLE}}$ for a fresh sid :

1. Simulate $\mathcal{F}_{\text{m} \times \text{RSOT}_{\text{Scalar}^{\ell+1}}}$ (Functionality 3.4) to the adversary:
 - (a) Receive $(\text{receiverInput}, \text{sid} \parallel \text{RSOT}, \mathbf{w}, \mathbf{S})$ from the adversary.
 - (b) Send the adversary $(\text{receiverOutput}, \text{sid} \parallel \text{RSOT}, \mathbf{w}, \mathbf{S})$.
2. Sample the matrix $\mathbf{Q} \in \text{Scalar}^{\text{m} \times (\ell+1)}$ uniformly at random. Send $\mathbf{Q}$ to the adversary.
3. Define the matrix $\mathbf{R}$ as follows: for every $j \in [\text{m}]$, $i \in [\ell+1]$:

$$\mathbf{R}_{j,i} = \mathbf{S}_{j,i} + w_j \cdot \mathbf{Q}_{j,i}.$$

4. Receive back $\chi = (\chi_1, \dots, \chi_\ell, 1)$ from the adversary.
5. Sample v uniformly at random.
6. Compute for every $j \in [\text{m}]$

$$u_j = \langle \chi, \mathbf{R}_j \rangle - w_j \cdot v.$$
7. Send $(\mathbf{u}, v)$ to the adversary.
8. Receive back $\mathbf{g}$.
9. Set $b = \langle \mathbf{g}, \mathbf{w} \rangle$.
10. Compute for every $i \in [\ell]$:

$$r_i = \sum_{j=1}^{\text{m}} g_j \cdot r_{j,i}$$

11. Send to the functionality $\mathcal{F}_{\text{RVOLE}}$ (Functionality 4.1) the command $(\text{receiverInput}, \text{sid}, b, \mathbf{r})$.

We show that the real and ideal are identically distributed via the following sequence of hybrids:

Hyb_{R1}: This is the real execution.

Hyb_{R2}: We change the experiment such that $\mathcal{F}_{\mathbf{m} \times \text{RSOT}_{\text{Scalar}^{\ell+1}}}$ and the honest sender is the same machine (the simulator). We also assume the existence of a trusted party for $\mathcal{F}_{\text{RVOLE}}$, and that the environment sends a command (**multiply**, **sid**) to the honest party; however, the output of the experiment consists of the view of the adversary and the output of the real honest sender.

Hyb_{R3}: In the $\mathcal{F}_{\text{RSOT}}$ -hybrid model, $\mathcal{F}_{\mathbf{m} \times \text{RSOT}_{\text{Scalar}^{\ell+1}}}$ sets for every $j \in [\mathbf{m}]$, $\mathbf{T}_j^{w_j} = \mathbf{S}_j$, and samples $\mathbf{T}_j^{1-w_j} \leftarrow \text{Scalar}^{\ell+1}$. It then gives the honest **Sender** the matrices $\mathbf{T}^0, \mathbf{T}^1$; The sender chooses $\mathbf{A}$ and sets $\mathbf{Q} = \mathbf{T}^0 - \mathbf{T}^1 + \mathbf{A}$. For every $j \in [\mathbf{m}]$ it therefore holds that

$$\mathbf{Q}_j = \mathbf{T}_j^0 - \mathbf{T}_j^1 + \mathbf{A}_j .$$

In this hybrid, instead of sampling $\mathbf{T}_j^{1-w_j}$ uniformly at random as above, the simulator samples $\mathbf{Q}_j$ uniformly at random, and extracts $\mathbf{T}_j^{1-w_j}$ given $\mathbf{T}_j^{w_j}, \mathbf{A}_j, \mathbf{Q}_j$ and $\mathbf{w}$.

Hyb_{R4}: Instead of sampling $\hat{\alpha}$ uniformly at random, sample v uniformly at random, and extract:

$$\hat{\alpha} = v - \chi_1 \cdot \alpha_1 - \dots - \chi_\ell \cdot \alpha_\ell .$$

Hyb_{R5}: We change the way the vector $\mathbf{u}$ is being computed. Instead of computing as in the real (using $\mathbf{T}$), we compute it as the receiver verifies the message. First, define:

$$\mathbf{R}_{j,i} = \mathbf{S}_{j,i} + w_j \cdot \mathbf{Q}_{j,i} .$$

For every $j \in [\mathbf{m}]$ compute:

$$u_j = \langle \chi, \mathbf{R}_j \rangle - w_j \cdot v = (\chi_1 \cdot r_{j,1} + \dots + \chi_\ell \cdot r_{j,\ell} + \hat{r}_j) - w_j \cdot v .$$

then set $\mathbf{u} = (u_1, \dots, u_{\mathbf{m}})$.

Hyb_{R6}: We change the way the output of the honest sender is computed. Instead of computing for every $i \in [\ell]$:

$$t_i = - \sum_{j=1}^{\mathbf{m}} g_j \cdot t_{j,i} ,$$

we define t_i as the trusted party computes it. Specifically, we set $(b, r_1, \dots, r_\ell)$ as in the simulation, that is, for every $i \in [\ell]$:

$$r_i = \sum_{j=1}^{\mathbf{m}} g_j \cdot r_{j,i}$$

Then, compute each t_i as:

$$t_i = \alpha_i \cdot b - r_i .$$

Hyb_{R7}: This is the ideal execution. Specifically, the output of the honest sender is chosen by the trusted party.

We now show that real and ideal are identically-distributed by showing that each one of the consecutive hybrids is identically distributed.

Hyb_{R1} and Hyb_{R2} are identical, as this is just syntactic change.

Hyb_{R2} and Hyb_{R3} are identically distributed. For every $j \in [m]$ there is one-to-one correspondence between $\mathbf{T}_j^{1-w_j}$ and $\mathbf{Q}_j$, given $\mathbf{T}_j^{w_j}$ and $\mathbf{A}_j$, described as $\mathbf{Q}_j = \mathbf{T}_j^0 - \mathbf{T}_j^1 + \mathbf{A}_j$.

Hyb_{R3} and Hyb_{R4} are identical. Specifically, there is one-to-one correspondence between v and $\hat{\alpha}$ given $(\alpha_1, \dots, \alpha_\ell)$ and χ . Specifically, $v = \langle \alpha, \chi \rangle$ and so

$$\hat{\alpha} = v - \chi_1 \cdot \alpha_1 - \dots - \chi_\ell \cdot \alpha_\ell .$$

Hyb_{R4} and Hyb_{R5} are identically distributed. According to $\mathcal{F}_{\text{RSOT}}$ it holds that:

$$\mathbf{S}_{j,i} = \mathbf{T}_{j,i}^0 + w_j \cdot (\mathbf{T}_{j,i}^1 - \mathbf{T}_{j,i}^0) .$$

Let

$$\mathbf{R}_{j,i} = \mathbf{S}_{j,i} + w_j \cdot \mathbf{Q}_{j,i} = \mathbf{T}_{j,i} + w_j \cdot \mathbf{A}_{j,i}$$

as follows from Eq. (3). Thus, for every $j \in [m], i \in [\ell]$ it holds that

$$r_{j,i} - t_{j,i} = w_j \cdot \alpha_i , \quad \hat{r}_j - \hat{t}_j = w_j \cdot \hat{\alpha} .$$

The honest sender computes u_i as:

$$\begin{aligned} u_j &= \langle \mathbf{T}_j, \chi \rangle \\ &= \chi_1 \cdot t_{j,1} + \dots + \chi_\ell \cdot t_{j,\ell} + \hat{t}_j \\ &= \chi_1 \cdot (r_{j,1} - w_j \cdot \alpha_1) + \dots + \chi_\ell \cdot (r_{j,\ell} - w_j \cdot \alpha_\ell) + (\hat{r}_j - w_j \cdot \hat{\alpha}) \\ &= \langle \chi, \mathbf{R}_j \rangle - w_j \cdot \langle \chi, \alpha \rangle \\ &= \langle \chi, \mathbf{R}_j \rangle - w_j \cdot v . \end{aligned}$$

which is exactly the way the simulator computes each u_j in **Hyb₅**.

Hyb₅ and Hyb₆ are identically distributed. In **Hyb₅**, the honest sender computes

$$t_i = - \sum_{j=1}^m g_j \cdot t_{j,i} .$$

In **Hyb_{R6}** we have:

$$t_i = \alpha_i \cdot b - \sum_{j=1}^m g_j \cdot r_{j,i}$$

Note that, $b = \sum_{j=1}^m g_j \cdot w_j$. Therefore:

$$\begin{aligned}
t_i &= \alpha_i \cdot b - \sum_{j=1}^m g_j \cdot r_{j,i} \\
&= \alpha_i \cdot \sum_{j=1}^m g_j \cdot w_j - \sum_{j=1}^m g_j \cdot r_{j,i} \\
&= \sum_{j=1}^m g_j \cdot (\alpha_i \cdot w_j - r_{j,i}) \\
&= - \sum_{j=1}^m g_j \cdot t_{j,i}
\end{aligned}$$

where the latter holds since $r_{j,i} - t_{j,i} = \alpha_i \cdot w_j$. This is exactly as t_i is computed in **Hyb_{R6}**.

Hyb_{R6} and Hyb_{R7} are identical. This is again just a syntactic change.

Concluding the proof. From combining all hybrids, we obtain that the real and ideal are completely identical. $\square$

B.2 Variant II: Compressing Rounds – Security Analysis for Protocol 4.2

We now perform a Fiat-Shamir-like transformation to reduce the number of rounds. We perform the following modifications, which bring us back to Protocol 4.2:

1. Instead of picking χ by the receiver, it is computed as $\mathcal{F}_{\text{ROM}}^{\text{Scalar}^\ell}(\text{sid}, \text{chi}, \mathbf{Q})$. This eliminates round 3 in the extended protocol, and now the sender can compute rounds 2,3,4 at once.
2. Next, the sender computes $H_u = \mathcal{F}_{\text{ROM}}^{2\kappa}(\text{sid}, \mathbf{u}, \mathbf{u})$ and sends H_u instead of $\mathbf{u}$ in its entirety.
3. Finally, the sender derives $\mathbf{g} = \mathcal{F}_{\text{ROM}}^{\text{Scalar}^m}(\text{sid}, \text{gadget}, m_2) = \mathcal{F}_{\text{ROM}}^{\text{Scalar}^m}(\text{sid}, \text{gadget}, (\mathbf{Q}, H_u, v))$ instead of receiving it from the receiver. This eliminates round 5 of the extended protocol, and now the sender can compute rounds 2–6 at once.

We now show how to modify the simulator for each one of those modifications.

Changes in the simulation – corrupted Sender. We follow the same steps of Simulator B.3, with the following modifications:

1. In Step 5, instead of picking $(\chi_1, \dots, \chi_\ell)$ uniformly at random by the simulator, the simulator simply computes $(\chi_1, \dots, \chi_\ell) = \mathcal{F}_{\text{ROM}}^{\text{Scalar}^\ell}(\text{sid}, \text{chi}, \mathbf{Q})$ as the honest **Receiver** in the real. Thus, χ is well-defined at the same point in the simulation as previously.
2. In Step 6, instead of receiving $(\mathbf{u}, v)$ from the adversary, the adversary sends just (H_u, v) . The simulator then scans the random oracle queries that the adversary makes and looks for a single query $(\text{sid}, \mathbf{u}, \mathbf{u})$ that results in H_u . If there is no unique query, it aborts. It then continues while using this unique $\mathbf{u}$ as if the adversary sent it explicitly, and $\mathbf{u}$ is well-defined at the same point in the simulation as previously.

3. In Step 9, instead of sampling $\mathbf{g}$ uniformly at random in Scalar^m , compute: $\mathbf{g} = \mathcal{F}_{\text{ROM}}^{\text{Scalar}^m}(\text{sid}, \text{gadget}, m_2)$, where $m_2 = (\mathbf{Q}, H_u, v)$. Note that the simulator already has those messages. Thus, $\mathbf{g}$ is well-defined at the same point as previously.

We then show how the above changes affect the simulation.

Picking χ by the random oracle: For this modification, the simulation fails with probability at most $(Q + 1) \cdot 2^{-\kappa}$, where Q bounds the number of queries that the adversary makes to the random oracle.

Specifically, in the $\mathcal{F}_{\text{RST}}\text{-random}$ oracle, the matrices $\mathbf{T}^0, \mathbf{T}^1$ are fixed. This implies that each $\mathbf{Q}$ uniquely defines $\mathbf{A} = \mathbf{Q} - \mathbf{T}^0 + \mathbf{T}^1$. The vector χ is chosen uniformly for each $\mathbf{Q}$. Claim B.8 shows that the probability that $(\mathbf{A}, \chi)$ is “bad” for a random χ is bounded by $2^{-\kappa}$. Thus, an adversary that makes at most Q queries to the random oracle succeeds in finding a pair $(\mathbf{A}, \chi)$ that is bad is bounded by $(Q + 1) \cdot 2^{-\kappa}$.

The property of χ being “not bad” is used in Claim B.7. Since χ is not bad with overwhelming probability, $|S| = 1$, and the values $(\alpha_1, \dots, \alpha_\ell)$ are well-defined.

Sending H_u instead of $\mathbf{u}$. This further introduces an error of $(Q^2 + 1) \cdot 2^{-2\kappa}$. There are two cases to consider: either the simulator does not find a query to the random oracle that results in H_u , or that it finds more than one:

- If no query, then the receiver queries a value $\mathbf{u}'$ that the sender did not query. Since H_u is fixed, the probability that $\mathcal{F}_{\text{ROM}}^{2\kappa}(\text{sid}, \mathbf{u}, \mathbf{u}') = H_u$ is $2^{-2\kappa}$.
- If there are multiple queries that lead to H_u , then the adversary has found a collision in the random oracle. The probability that two specific queries collide is less than $2^{-2\kappa}$. The probability of finding any collision is bounded by $Q^2 \cdot 2^{-2\kappa}$.

Picking $\mathbf{g}$ by the random oracle. This modification leads to increasing the number of OTs from $\log q + 3s$ (≈ 436) to $\log q + 2 \cdot \kappa + 3s$ (≈ 692).

Specifically, the adversary can now, given a candidate $(\mathbf{Q}, H_u, v)$, after fixing the corresponding extracted $\mathbf{u}$ except for a collision error probability, the adversary can query the random oracle to retrieve $\mathbf{g}$ and decide whether to proceed with that $\mathbf{g}$ or not. Each $(\mathbf{Q}, \mathbf{u}, v)$ defines a set of constraints $\mathbf{c}$. For each such quadruple, there is a unique $\mathbf{g}$ that will be chosen by the random oracle. The adversary can try many different quadruples. Claim 5.5 captures exactly this scenario and shows that when $m \geq \log q + 3s + 2\kappa$, the statistical distance between $\langle \mathbf{g}, \mathbf{w} \rangle$ and the uniform distribution over Scalar is at most $Q \cdot 2^{-\kappa} + 2^{-s}$, for an adversary that makes Q queries to the random oracle.

Changes in the simulation – corrupted Receiver. We follow the same steps of Simulator B.15, with the following modifications:

1. In Step 4, instead of receiving χ from the adversary, the simulator simply computes $(\chi_1, \dots, \chi_\ell) = \mathcal{F}_{\text{ROM}}^{\text{Scalar}^\ell}(\text{sid}, \text{chi}, \mathbf{Q})$ and sets $\chi = (\chi_1, \dots, \chi_\ell, 1)$.
2. Compute $H_u = \mathcal{F}_{\text{ROM}}^{2\kappa}(\text{sid}, \mathbf{u}, \mathbf{u})$.
3. In Step 7, instead of sending $(\mathbf{u}, v)$ to the adversary and receiving back $\mathbf{g}$, the adversary stores (H_u, v) .

4. In Step 8, instead of receiving $\mathbf{g}$ from the adversary, compute: $\mathbf{g} = \mathcal{F}_{\text{ROM}}^{\text{Scalar}^m}(\text{sid}, \text{gadget}, (\mathbf{Q}, H_u, v))$.
5. The simulator computes $b = \langle \mathbf{g}, \mathbf{w} \rangle$.
6. The simulator sends the adversary the message $(\mathbf{Q}, H_u, v)$ and sends the $\mathcal{F}_{\text{RVOLE}}$ the command $(\text{receiverInput}, \text{sid}, b, \mathbf{r})$.

Simulator B.15 defines ideal and real that are identical *for every* χ and *for every* $\mathbf{g}$. Therefore, they are also identical for those particular choices of χ and $\mathbf{g}$ by the random oracle. Moreover, we show that the view of the adversary is identical in the real and ideal when in both it received $\mathbf{u}$ in its entirety; therefore, it will remain identical when instead it receives in both $H_u = \mathcal{F}_{\text{ROM}}^{2\kappa}(\text{sid}, \mathbf{u}, \mathbf{u})$.

We conclude:

Theorem B.16. *Assuming that:*

1. $\log q \geq \kappa + 2 \log m$,
2. $m \geq \log q + 2 \cdot \kappa + 3 \cdot s$;

then Protocol 4.2 securely realizes $\mathcal{F}_{\text{RVOLE}}$ (Functionality 4.1) in the $\mathcal{F}_{\text{ROM}}\text{-}\mathcal{F}_{\text{RSOT}}$ -hybrid model.

B.3 Variant III: The Protocol of [DKLs24]

In this variant, we change the protocol so that $\mathbf{g}$ is chosen at the onset of the protocol, and is defined as $\mathbf{g} = \mathcal{F}_{\text{ROM}}^{\text{Scalar}^m}(\text{sid})$. Both the **Sender** and **Receiver** can compute it as soon as they have sid . At the end of the first round, the **Receiver** can already compute $b = \langle \mathbf{g}, \mathbf{w} \rangle$. We have:

Theorem B.17. *If*

1. $\log q \geq \kappa + 2 \log m$,
2. $m \geq \log q + 7 \cdot s + 2 \log(s + 1) + 2s \cdot \log(e \cdot m/s)$;

then Protocol 4.2 (when $\mathbf{g}$ is picked at the onset of the protocol) securely realizes $\mathcal{F}_{\text{RVOLE}}$ (Functionality 4.1) in the $\mathcal{F}_{\text{ROM}}\text{-}\mathcal{F}_{\text{RSOT}}$ -hybrid model.

Proof. The only difference is in the last hybrid of a corrupted **Sender**. In this hybrid, we replace b from $\langle \mathbf{g}, \mathbf{w} \rangle$ to be uniform in **Scalar**. The hash function $\mathbf{g}$ is already fixed (it was chosen on the onset of the computation), and the adversary might choose the set of constraints $\mathbf{c}$ is a function of $\mathbf{g}$. Claim 5.7 shows that even though $\mathbf{g}$ is fixed, when m is sufficiently large there is no constraint $\mathbf{c}$ for which the statistical distance between $b = \langle \mathbf{g}, \mathbf{w} \rangle$ and a random $b \leftarrow \text{Scalar}$ is large. $\square$

C A Full Proof for the Two-Party Signing Protocol

Theorem C.1 (Theorem 6.3, restated). *Protocol 6.2 securely realizes $\mathcal{F}_{\text{Sign}}^{\text{ECDSA}}$ (Functionality 6.1) in the $(F_{\text{com}}, \mathcal{F}_{\text{RVOLE}})$ -hybrid model (Functionalities 3.3, 4.1).*

Proof. We start with correctness for the case that both parties are honest.

Correctness. Recall that:

$$\begin{aligned}\llbracket r_{1-i} \cdot \beta_i \rrbracket_i &= \llbracket \tilde{r}_{1-i} \cdot \beta_i \rrbracket_i, \\ \llbracket r_{1-i} \cdot \beta_i \rrbracket_{1-i} &= \gamma_{r_{1-i}} \cdot \beta_i + \llbracket \tilde{r}_{1-i} \cdot \beta_i \rrbracket_{1-i}, \\ \gamma_{r_{1-i}} &= r_{1-i} - \tilde{r}_{1-i}.\end{aligned}$$

Thus, the shifted shares satisfy:

$$\begin{aligned}\llbracket r_{1-i} \cdot \beta_i \rrbracket_i + \llbracket r_{1-i} \cdot \beta_i \rrbracket_{1-i} &= \llbracket \tilde{r}_{1-i} \cdot \beta_i \rrbracket_i + (\gamma_{r_{1-i}} \cdot \beta_i + \llbracket \tilde{r}_{1-i} \cdot \beta_i \rrbracket_{1-i}) \\ &= \llbracket \tilde{r}_{1-i} \cdot \beta_i \rrbracket_i + (r_{1-i} - \tilde{r}_{1-i})\beta_i + \llbracket \tilde{r}_{1-i} \cdot \beta_i \rrbracket_{1-i} \\ &= \tilde{r}_{1-i} \cdot \beta_i + (r_{1-i} - \tilde{r}_{1-i})\beta_i \\ &= r_{1-i} \cdot \beta_i.\end{aligned}$$

In a similar manner, $\llbracket r_i \cdot \beta_{1-i} \rrbracket_i + \llbracket r_i \cdot \beta_{1-i} \rrbracket_{1-i} = r_i \cdot \beta_{1-i}$. Let $r = r_0 + r_1$ and $\beta = \beta_0 + \beta_1$. We have:

$$\begin{aligned}u_0 + u_1 &= r_0 \cdot \beta_0 + \llbracket r_0 \cdot \beta_1 \rrbracket_0 + \llbracket r_1 \cdot \beta_0 \rrbracket_0 \\ &\quad + r_1 \cdot \beta_1 + \llbracket r_0 \cdot \beta_1 \rrbracket_1 + \llbracket r_1 \cdot \beta_0 \rrbracket_1 \\ &= (r_0 + r_1) \cdot (\beta_0 + \beta_1) \\ &= r \cdot \beta.\end{aligned}$$

In a similar manner, let $\mathbf{sk} = \mathbf{sk}_0 + \mathbf{sk}_1$, we have that $v_0 + v_1 = \mathbf{sk} \cdot \beta$. Thus,

$$\begin{aligned}w_0 + w_1 &= (\mathbf{H}(m) \cdot \beta_0 + r^\times \cdot v_0) \\ &\quad + (\mathbf{H}(m) \cdot \beta_1 + r^\times \cdot v_1) \\ &= \mathbf{H}(m) \cdot \beta + r^\times \cdot \mathbf{sk} \cdot \beta \\ &= \beta \cdot (\mathbf{H}(m) + r^\times \cdot \mathbf{sk}).\end{aligned}$$

Now let $w = w_0 + w_1$ and $u = u_0 + u_1 = r \cdot \beta$, and we have:

$$s = w/u = r^{-1} \cdot (\mathbf{H}(m) + r^\times \cdot \mathbf{sk}).$$

Simulation. We assume without loss of generality that the adversary controls 0, and the honest party is 1.

Simulator C.2: Simulating Malicious P_0

1. Receive from the adversary its input to the commitment scheme – R_0 .
2. Simulate a receipt from F_{com} .
3. Receive from the adversary its input to the RVOLE –

$$\beta_0, (\llbracket \tilde{r}_1 \cdot \beta_0 \rrbracket_0, \llbracket \tilde{\mathbf{sk}}_1 \cdot \beta_0 \rrbracket_0), \quad \text{and} \quad (\tilde{r}_0, \tilde{\mathbf{sk}}_0, \llbracket \tilde{r}_0 \cdot \beta_1 \rrbracket_0, \llbracket \tilde{\mathbf{sk}}_0 \cdot \beta_1 \rrbracket_0).$$

4. Receive from the trusted party $(\mathbf{pk}, \mathbf{pk}_0, \mathbf{pk}_1)$, the signature σ and the message m . Parse $\sigma = (r^\times, s)$, and compute

$$R = (s^{-1} \cdot \mathbf{H}(m)) \cdot G + (s^{-1} \cdot r^\times) \cdot \mathbf{pk}.$$

5. Set $R_1 = R - R_0$.
6. Sample γ_{r_1} and $\gamma_{\mathbf{sk}_1}$ as Scalars.
7. Compute

$$\begin{aligned}\llbracket r_1 \cdot \beta_0 \rrbracket_0 &= \gamma_{r_1} \cdot \beta_0 + \llbracket \tilde{r}_1 \cdot \beta_0 \rrbracket_0, \\ \llbracket \mathbf{sk}_1 \cdot \beta_0 \rrbracket_0 &= \gamma_{\mathbf{sk}_1} \cdot \beta_0 + \llbracket \tilde{\mathbf{sk}}_1 \cdot \beta_0 \rrbracket_0.\end{aligned}$$

8. Compute

$$\begin{aligned}\Gamma_{r_1} &= \beta_0 \cdot R_1 - \llbracket r_1 \cdot \beta_0 \rrbracket_0 \cdot G, \\ \Gamma_{\mathbf{sk}_1} &= \beta_0 \cdot \mathbf{pk}_1 - \llbracket \mathbf{sk}_1 \cdot \beta_0 \rrbracket_0 \cdot G.\end{aligned}$$

9. Simulate P_1 sending the adversary

$$(\Gamma_{r_1}, \Gamma_{\mathbf{sk}_1}, \gamma_{r_1}, \gamma_{\mathbf{sk}_1})$$

and the opening of R_1 .

10. Receive from the adversary

$$(\Gamma_{r_0}, \Gamma_{\mathbf{sk}_0}, \gamma_{r_0}, \gamma_{\mathbf{sk}_0})$$

11. Extract:

$$\begin{aligned}r_0 &= \gamma_{r_0} + \tilde{r}_0 \\ \mathbf{sk}_0 &= \gamma_{\mathbf{sk}_0} + \tilde{\mathbf{sk}}_0.\end{aligned}$$

12. Re-name the shares:

$$\begin{aligned}\llbracket r_0 \cdot \beta_1 \rrbracket_0 &:= \llbracket \tilde{r}_0 \cdot \beta_1 \rrbracket_0 \\ \llbracket \mathbf{sk}_0 \cdot \beta_1 \rrbracket_0 &:= \llbracket \tilde{\mathbf{sk}}_0 \cdot \beta_1 \rrbracket_0.\end{aligned}$$

13. Verify that:

- (a) $\Gamma_{r_0}, \Gamma_{\mathbf{sk}_0}$ are points on the curve. If not, abort.
- (b) Verify that $\Gamma_{r_0} = \llbracket r_0 \cdot \beta_1 \rrbracket_0 \cdot G$, and that $\Gamma_{\mathbf{sk}_0} = \llbracket \mathbf{sk}_0 \cdot \beta_1 \rrbracket_0 \cdot G$; Otherwise, abort.
- (c) Verify that $\mathbf{sk}_0 \cdot G = \mathbf{pk}_0$, and that $r_0 \cdot G = R_0$. Otherwise, abort.

14. Let $r^\times$ be the x -coordinate of R . Compute the values that $\mathcal{A}$ is supposed to send at the end of this round:

$$\begin{aligned}u_0 &= r_0 \cdot \beta_0 + \llbracket r_0 \cdot \beta_1 \rrbracket_0 + \llbracket r_1 \cdot \beta_0 \rrbracket_0, \\ v_0 &= \mathbf{sk}_0 \cdot \beta_0 + \llbracket \mathbf{sk}_0 \cdot \beta_1 \rrbracket_0 + \llbracket \mathbf{sk}_1 \cdot \beta_0 \rrbracket_0, \\ w_0 &= H(m) \cdot \beta_0 + r^\times \cdot v_0.\end{aligned}$$

15. Sample u_1 . Let $u = u_0 + u_1$.
16. Compute $w = u \cdot s$. Compute $w_1 = w - w_0$.
17. Send the adversary (u_1, w_1) .
18. Receive from the adversary (u'_0, w'_0) .
19. Compute $u' = u_1 + u'_0$ and $w' = w'_0 + w_1$, and set $s' = w'/u'$.

20. Set $\sigma' = (r^x, s')$. Run $\text{Vrfy}_{\text{pk}}(m, \sigma')$. If the result is 1, send OK to the trusted party. Otherwise, abort.
-

We show that the real and ideal are indistinguishable via a sequence of hybrid experiments.

- **Hyb₀**: This is the real execution. The output of this hybrid is the view of the adversary together with the output of the honest P_1 .
- **Hyb₁**: We combine all the ideal functionalities $\mathcal{F}_{\text{RVOLE}}, F_{\text{com}}$ to one global simulator together with the real P_1 .
- **Hyb₂**: In this hybrid, instead of P_1 verifying as in Steps 2d, 2e in the real, we verify as in Step 11, and 13 in the simulation. Among other things, verifying that
 1. Verify that $\Gamma_{r_0} = \llbracket r_0 \cdot \beta_1 \rrbracket_0 \cdot G$, and that $\Gamma_{\text{sk}_0} = \llbracket \text{sk}_0 \cdot \beta_1 \rrbracket_0 \cdot G$.
 2. Verify that $\text{sk}_0 \cdot G = \text{pk}_0$, and that $r_0 \cdot G = R_0$.

Along the way, extract r_0, sk_0 .

- **Hyb₃**: In this hybrid, we change γ_{r_1} and γ_{sk_1} . Instead of picking $\tilde{r}_1$ and $\tilde{\text{sk}}_1$ uniformly at random, and then computing

$$\begin{aligned}\gamma_{r_1} &= r_1 - \tilde{r}_1, \\ \gamma_{\text{sk}_1} &= \text{sk}_1 - \tilde{\text{sk}}_1,\end{aligned}$$

the experiment first picks γ_{r_1} and γ_{sk_1} uniformly at random, and then sets

$$\begin{aligned}\tilde{r}_1 &= r_1 - \gamma_{r_1} \\ \tilde{\text{sk}}_1 &= \text{sk}_1 - \gamma_{\text{sk}_1}.\end{aligned}$$

This is possible as the output of the adversary in the RVOLE is chosen by the adversary (and is independent of $\tilde{r}_1, \tilde{\text{sk}}_1$).

- **Hyb₄**: In this hybrid, we change Γ_{r_1} and Γ_{sk_1} . Instead of computing

$$\Gamma_{r_1} = \llbracket r_1 \cdot \beta_0 \rrbracket_1 \cdot G, \quad \Gamma_{\text{sk}_1} = \llbracket \text{sk}_1 \cdot \beta_0 \rrbracket_1 \cdot G,$$

i.e., using the shares of P_1 as in the real, we compute as in the simulation (using the shares of P_0). Namely, compute

$$\begin{aligned}\llbracket r_1 \cdot \beta_0 \rrbracket_0 &= \gamma_{r_1} \cdot \beta_0 + \llbracket \tilde{r}_1 \cdot \beta_0 \rrbracket_0, \\ \llbracket \text{sk}_1 \cdot \beta_0 \rrbracket_0 &= \gamma_{\text{sk}_1} \cdot \beta_0 + \llbracket \tilde{\text{sk}}_1 \cdot \beta_0 \rrbracket_0.\end{aligned}$$

and then

$$\begin{aligned}\Gamma_{r_1} &= \beta_0 \cdot R_1 - \llbracket r_1 \cdot \beta_0 \rrbracket_0 \cdot G, \\ \Gamma_{\text{sk}_1} &= \beta_0 \cdot \text{pk}_1 - \llbracket \text{sk}_1 \cdot \beta_0 \rrbracket_0 \cdot G.\end{aligned}$$

Note that the simulator now does not use (r_1, sk_1) , nor its shares from $\mathcal{F}_{\text{RVOLE}}$ to compute the message $(\Gamma_{r_1}, \Gamma_{\text{sk}_1}, \gamma_{r_1}, \gamma_{\text{sk}_1})$.

- **Hyb₅**: In this hybrid, we change how u_1, β_1 are defined. In **Hyb₄**, the $\mathcal{F}_{\text{RVOLE}}$ in which the adversary is the **Sender**, the functionality $\mathcal{F}_{\text{RVOLE}}$ picks β_1 uniformly at random, and computes the shares of P_1 , and then

$$u_1 = r_1 \cdot \beta_1 + \llbracket r_1 \cdot \beta_0 \rrbracket_1 + \llbracket r_0 \cdot \beta_1 \rrbracket_1 .$$

In this hybrid, we use $r = r_0 + r_1$ where r_0 was extracted by the simulator, and r_1 was generated by the honest P_1 . We compute

$$u_0 = r_0 \cdot \beta_0 + \llbracket r_0 \cdot \beta_1 \rrbracket_0 + \llbracket r_1 \cdot \beta_0 \rrbracket_0$$

that the adversary is supposed to hold. Then, instead of picking β_1 uniformly at random, we pick u_1 uniformly at random, and set

$$\beta_1 = (u_0 + u_1) \cdot r^{-1} - \beta_0 .$$

We then use β_1 to derive the shares of the honest P_1 in the $\mathcal{F}_{\text{RVOLE}}$, and compute w_1 as in the real.

- **Hyb₆**: In this hybrid, we change the way w_1 is constructed. Instead of computing v_1, w_1 as in the protocol, we first compute the shares that the adversary is supposed to hold:

$$\begin{aligned} v_0 &= \text{sk}_0 \cdot \beta_0 + \llbracket \text{sk}_0 \cdot \beta_1 \rrbracket_0 + \llbracket \text{sk}_1 \cdot \beta_0 \rrbracket_0 , \\ w_0 &= \text{H}(m) \cdot \beta_0 + r^x \cdot v_0 . \end{aligned}$$

Then, we compute $s = r^{-1} \cdot (\text{H}(m) + \text{sk} \cdot r^x)$ using the extracted sk and extracted nonce r . Then, instead of computing w_1 as

$$w_1 = \text{H}(m) \cdot \beta_1 + r^x \cdot v_1 ,$$

it is computed as:

$$w_1 = s \cdot (u_0 + u_1) - w_0 .$$

- **Hyb₇**: In this hybrid, instead of computing $R_1 = r_1 \cdot G$, then the simulator receives σ from the trusted party. Moreover, it defines $R_1 = R - R_0$, where R is reconstructed from the signature σ . Note that r_1 is not used in the simulator. Also sk_1 is not used by the simulator (and instead it is used by the trusted party to compute the signature σ).
- **Hyb₈**: We change the output of the honest party. Instead of computing $\sigma = (r^x, s)$ as in the real, it receives the output from the trusted party.

We proceed to show that every pair of consecutive hybrids is identical or statistically-close.

Hyb₁ and Hyb₂ are statistically-close. In both hybrids, the first message that the adversary receives in the signing phase is computed identically. When the adversary sends its message

$$(\Gamma_{r_0}, \Gamma_{\text{sk}_0}, \gamma_{r_0}, \gamma_{\text{sk}_0}), \quad R_0$$

Hyb₁ verifies that $\Gamma_{r_0}, \Gamma_{\text{sk}_0}, R_0$ are points on the curve. Moreover, it re-defines the following shares:

$$\begin{aligned} \llbracket r_0 \cdot \beta_1 \rrbracket_1 &= \gamma_{r_0} \cdot \beta_1 + \llbracket \tilde{r}_0 \cdot \beta_1 \rrbracket_1, \\ \llbracket \text{sk}_0 \cdot \beta_1 \rrbracket_1 &= \gamma_{\text{sk}_0} \cdot \beta_1 + \llbracket \tilde{\text{sk}}_0 \cdot \beta_1 \rrbracket_1. \end{aligned}$$

and verifies the following equations

$$\begin{aligned}\llbracket r_0 \cdot \beta_1 \rrbracket_1 \cdot G &= \beta_1 \cdot R_0 - \Gamma_{r_0}, \\ \llbracket \text{sk}_0 \cdot \beta_1 \rrbracket_1 \cdot G &= \beta_1 \cdot \text{pk}_0 - \Gamma_{\text{sk}_0}.\end{aligned}$$

In **Hyb₂**, the simulator verifies that $\Gamma_{r_0}, \Gamma_{\text{sk}_0}, R_0$ are points on the curve. Moreover, it extracts r_0 and sk_0 and verifies that they are correct, and in addition,

$$\begin{aligned}R_0 &= r_0 \cdot G = (\gamma_{r_0} + \tilde{r}_0) \cdot G \\ \text{pk}_0 &= \text{sk}_0 \cdot G = (\gamma_{\text{sk}_0} + \tilde{\text{sk}}_0) \cdot G \\ \Gamma_{r_0} &= \llbracket r_0 \cdot \beta_1 \rrbracket_0 \cdot G = \llbracket \tilde{r}_0 \cdot \beta_1 \rrbracket_0 \cdot G, \\ \Gamma_{\text{sk}_0} &= \llbracket \text{sk}_0 \cdot \beta_1 \rrbracket_0 \cdot G = \llbracket \tilde{\text{sk}}_0 \cdot \beta_1 \rrbracket_0 \cdot G\end{aligned}$$

We show that if **Hyb₁** aborts then **Hyb₂** aborts. Specifically, assume that

$$(\gamma_{r_0} \cdot \beta_1 + \llbracket \tilde{r}_0 \cdot \beta_1 \rrbracket_1) \cdot G \neq \beta_1 \cdot R_0 - \Gamma_{r_0}$$

If $R_0 \neq (\gamma_{r_0} + \tilde{r}_0) \cdot G$ then **Hyb₂** aborts. Otherwise, i.e., $R_0 = (\gamma_{r_0} + \tilde{r}_0) \cdot G$, then:

$$\begin{aligned}(\gamma_{r_0} \cdot \beta_1 + \llbracket \tilde{r}_0 \cdot \beta_1 \rrbracket_1) \cdot G &\neq \beta_1 \cdot R_0 - \Gamma_{r_0} \\ (\gamma_{r_0} \cdot \beta_1 + \llbracket \tilde{r}_0 \cdot \beta_1 \rrbracket_1) \cdot G &\neq \beta_1 \cdot (\gamma_{r_0} + \tilde{r}_0) \cdot G - \Gamma_{r_0} \\ \llbracket \tilde{r}_0 \cdot \beta_1 \rrbracket_1 \cdot G &\neq \beta_1 \cdot \tilde{r}_0 \cdot G - \Gamma_{r_0} \\ \Gamma_{r_0} &\neq \beta_1 \cdot \tilde{r}_0 \cdot G - \llbracket \tilde{r}_0 \cdot \beta_1 \rrbracket_1 \cdot G \\ \Gamma_{r_0} &\neq \llbracket \tilde{r}_0 \cdot \beta_1 \rrbracket_0 \cdot G.\end{aligned}$$

and thus **Hyb₂** also aborts. A similar argument holds for the equations regarding sk_0 . This implies that the checks in **Hyb₂** are more robust than the checks in **Hyb₁**.

A difference between **Hyb₂** and **Hyb₁** might occur when **Hyb₂** aborts, but **Hyb₁** does not abort. If **Hyb₁** does not abort, then

$$(\gamma_{r_0} \cdot \beta_1 + \llbracket \tilde{r}_0 \cdot \beta_1 \rrbracket_1) \cdot G = \beta_1 \cdot R_0 - \Gamma_{r_0} \quad (4)$$

If $R_0 = (\gamma_{r_0} + \tilde{r}_0) \cdot G$, then this implies that $\Gamma_{r_0} = \llbracket \tilde{r}_0 \cdot \beta_1 \rrbracket_0 \cdot G$ as well. Therefore, if there is a difference it must hold that both

$$R_0 \neq (\gamma_{r_0} + \tilde{r}_0) \cdot G, \quad \text{and} \quad \Gamma_{r_0} \neq \llbracket \tilde{r}_0 \cdot \beta_1 \rrbracket_0 \cdot G,$$

while Eq. (4) holds. We can write, for some non-zero Δ, Δ' :

$$R_0 = (\gamma_{r_0} + \tilde{r}_0) \cdot G + \Delta, \quad \text{and} \quad \Gamma_{r_0} = \llbracket \tilde{r}_0 \cdot \beta_1 \rrbracket_0 \cdot G + \Delta'. \quad (5)$$

Applying those in Eq. (4):

$$\begin{aligned}(\gamma_{r_0} \cdot \beta_1 + \llbracket \tilde{r}_0 \cdot \beta_1 \rrbracket_1) \cdot G &= \beta_1 \cdot R_0 - \Gamma_{r_0} \\ (\gamma_{r_0} \cdot \beta_1 + \llbracket \tilde{r}_0 \cdot \beta_1 \rrbracket_1) \cdot G &= \beta_1 \cdot ((\gamma_{r_0} + \tilde{r}_0) \cdot G + \Delta) - \llbracket \tilde{r}_0 \cdot \beta_1 \rrbracket_0 \cdot G - \Delta' \\ \llbracket \tilde{r}_0 \cdot \beta_1 \rrbracket_1 \cdot G &= (\beta_1 \cdot \tilde{r}_0) \cdot G + \beta_1 \cdot \Delta - \llbracket \tilde{r}_0 \cdot \beta_1 \rrbracket_0 \cdot G - \Delta' \\ (\llbracket \tilde{r}_0 \cdot \beta_1 \rrbracket_1 + \llbracket \tilde{r}_0 \cdot \beta_1 \rrbracket_0) \cdot G &= (\beta_1 \cdot \tilde{r}_0) \cdot G + \beta_1 \cdot \Delta - \Delta' \\ \Delta' &= \beta_1 \cdot \Delta\end{aligned}$$

where it follows from the properties of the RVOLE that $\llbracket \tilde{r}_0 \cdot \beta_1 \rrbracket_1 + \llbracket \tilde{r}_0 \cdot \beta_1 \rrbracket_0 = \tilde{r}_0 \cdot \beta_1$. Since β_1 is independent of the view of the adversary in that round and is chosen uniformly at random, the probability that the adversary outputs R_0, Γ_{r_0} such that both Eq. (5) and Eq. (4) hold occur with probability at most $1/|\text{Scalar}|$. A similar argument also holds for equations that are related to sk_0 .

Hyb₂ and Hyb₃ are identical. In **Hyb₂**, the honest P_1 lets $\mathcal{F}_{\text{RVOLE}}$ to pick $\tilde{r}_1$ uniformly at random, and then sets $\gamma_{r_1} = r_1 - \tilde{r}_1$. In **Hyb₃**, the experiment first samples $\gamma_{r_1}, \gamma_{\text{sk}_1}$, and then sets $\tilde{r}_1 = r_1 - \gamma_{r_1}$. Clearly, there is a one-to-one correspondence between those two values, given r_1 . As such, the distribution is identical. The same holds for $\text{sk}_1, \gamma_{\text{sk}_1}$.

Hyb₃ and Hyb₄ are identical. In **Hyb₃**, the honest P_1 computes

$$\Gamma_{r_1} = \llbracket r_1 \cdot \beta_0 \rrbracket_1 \cdot G = \llbracket \tilde{r}_1 \cdot \beta_0 \rrbracket_1 \cdot G$$

where $\llbracket \tilde{r}_1 \cdot \beta_0 \rrbracket_1$ is computed from the $\mathcal{F}_{\text{RVOLE}}$: The adversary provides $\llbracket \tilde{r}_1 \cdot \beta_0 \rrbracket_0$ and β_0 , the $\mathcal{F}_{\text{RVOLE}}$ uses $\tilde{r}_1$, and computes

$$\llbracket \tilde{r}_1 \cdot \beta_0 \rrbracket_1 = \tilde{r}_1 \cdot \beta_0 - \llbracket \tilde{r}_1 \cdot \beta_0 \rrbracket_0 .$$

Moreover, it holds that $\gamma_{r_1} = r_1 - \tilde{r}_1$. Thus,

$$\begin{aligned} \Gamma_{r_1} &= \llbracket \tilde{r}_1 \cdot \beta_0 \rrbracket_1 \cdot G \\ &= (\tilde{r}_1 \cdot \beta_0 - \llbracket \tilde{r}_1 \cdot \beta_0 \rrbracket_0) \cdot G \\ &= ((r_1 - \gamma_{r_1}) \cdot \beta_0 - \llbracket \tilde{r}_1 \cdot \beta_0 \rrbracket_0) \cdot G \\ &= (r_1 \cdot \beta_0 - \gamma_{r_1} \cdot \beta_0 - \llbracket \tilde{r}_1 \cdot \beta_0 \rrbracket_0) \cdot G \\ &= (r_1 \cdot \beta_0 - \llbracket r_1 \cdot \beta_0 \rrbracket_0) \cdot G \\ &= \beta_0 \cdot R_1 - \llbracket r_1 \cdot \beta_0 \rrbracket_0 \cdot G , \end{aligned}$$

as in **Hyb₄**. A similar argument holds for Γ_{sk_1} .

So far, the hybrids we defined completely change the first message

$$(\Gamma_{r_1}, \Gamma_{\text{sk}_1}, \gamma_{r_1}, \gamma_{\text{sk}_1})$$

that the adversary receives, and basing them on random values and its very own shares. Before modifying the second message and defining the consecutive hybrids, we first prove the following Claim:

Claim C.3. *Let r_0, r_1 be the nonces used for R_0, R_1 , and sk_0, sk_1 be the shares of pk_0, pk_1 . If there is no abort up to **Hyb₄**, it holds that:*

$$\begin{aligned} \llbracket r_0 \cdot \beta_1 \rrbracket_0 + \llbracket r_0 \cdot \beta_1 \rrbracket_1 &= r_0 \cdot \beta_1 & \llbracket r_1 \cdot \beta_0 \rrbracket_0 + \llbracket r_1 \cdot \beta_0 \rrbracket_1 &= r_1 \cdot \beta_0 \\ \llbracket \text{sk}_0 \cdot \beta_1 \rrbracket_0 + \llbracket \text{sk}_0 \cdot \beta_1 \rrbracket_1 &= \text{sk}_0 \cdot \beta_1 & \llbracket \text{sk}_1 \cdot \beta_0 \rrbracket_0 + \llbracket \text{sk}_1 \cdot \beta_0 \rrbracket_1 &= \text{sk}_1 \cdot \beta_0 \end{aligned}$$

Proof. We separate between the two instances of the $\mathcal{F}_{\text{RVOLE}}$. For the shares in which the adversary is the **Sender**, it holds that:

1. The adversary sent $\tilde{r}_0, \tilde{\text{sk}}_0$ to $\mathcal{F}_{\text{RVOLE}}$, and message $\gamma_{r_0}, \gamma_{\text{sk}_0}$ in the first round, such that $r_0 = \tilde{r}_0 + \gamma_{r_0}$ must be the pre-image of R_0 ; Likewise, the extracted $\text{sk}_0 = \tilde{\text{sk}}_0 + \gamma_{\text{sk}_0}$ is correct. Otherwise, the simulator aborts.

2. Denote the shares that the adversary sent to $\mathcal{F}_{\text{RVOLE}}$ as

$$\llbracket r_0 \cdot \beta_1 \rrbracket_0 = \llbracket \tilde{r}_0 \cdot \beta_1 \rrbracket_0 \quad \llbracket \text{sk}_0 \cdot \beta_1 \rrbracket_0 = \llbracket \tilde{\text{sk}}_0 \cdot \beta_1 \rrbracket_0 .$$

3. The honest P_1 (or the simulator) defines:

$$\llbracket r_0 \cdot \beta_1 \rrbracket_1 = \llbracket \tilde{r}_0 \cdot \beta_1 \rrbracket_1 + \gamma_{r_0} \cdot \beta_1 .$$

4. As such, it must hold that:

$$\begin{aligned} \llbracket r_0 \cdot \beta_1 \rrbracket_0 + \llbracket r_0 \cdot \beta_1 \rrbracket_1 &= \llbracket \tilde{r}_0 \cdot \beta_1 \rrbracket_0 + \llbracket \tilde{r}_0 \cdot \beta_1 \rrbracket_1 + \gamma_{r_0} \cdot \beta_1 \\ &= \tilde{r}_0 \cdot \beta_1 + \gamma_{r_0} \cdot \beta_1 \\ &= \tilde{r}_0 \cdot \beta_1 + (r_0 - \tilde{r}_0) \cdot \beta_1 \\ &= r_0 \cdot \beta_1 . \end{aligned}$$

Likewise,

$$\llbracket \text{sk}_0 \cdot \beta_1 \rrbracket_0 + \llbracket \text{sk}_0 \cdot \beta_1 \rrbracket_1 = \text{sk}_0 \cdot \beta_1 .$$

The following holds with respect to $\mathcal{F}_{\text{RVOLE}}$ for which the adversary is the **Receiver**:

1. The adversary inputs $\beta_0, \llbracket \tilde{r}_1 \cdot \beta_0 \rrbracket_0, \llbracket \tilde{\text{sk}}_1 \cdot \beta_0 \rrbracket_0$ to $\mathcal{F}_{\text{RVOLE}}$.
2. The modified simulator picked $\gamma_{r_1}, \gamma_{\text{sk}_1}$ uniformly at random, and sets $\tilde{r}_1 = r_1 - \gamma_{r_1}$, $\tilde{\text{sk}}_1 = \text{sk}_1 - \gamma_{\text{sk}_1}$.
3. The $\mathcal{F}_{\text{RVOLE}}$ is supposed to give the following outputs to P_1 :

$$\llbracket \tilde{r}_1 \cdot \beta_0 \rrbracket_1 = \tilde{r}_1 \cdot \beta_0 - \llbracket \tilde{r}_1 \cdot \beta_0 \rrbracket_0 , \quad \llbracket \tilde{\text{sk}}_1 \cdot \beta_0 \rrbracket_1 = \tilde{\text{sk}}_1 \cdot \beta_0 - \llbracket \tilde{\text{sk}}_1 \cdot \beta_0 \rrbracket_0$$

4. The simulator renames $\llbracket r_1 \cdot \beta_0 \rrbracket_1 = \llbracket \tilde{r}_1 \cdot \beta_0 \rrbracket_1$ and $\llbracket \text{sk}_1 \cdot \beta_0 \rrbracket_1 = \llbracket \tilde{\text{sk}}_1 \cdot \beta_0 \rrbracket_1$.
5. The simulator also defines the values that the adversary can compute and is supposed to hold:

$$\llbracket r_1 \cdot \beta_0 \rrbracket_0 = \llbracket \tilde{r}_1 \cdot \beta_0 \rrbracket_0 + \beta_0 \cdot \gamma_{r_1} , \quad \llbracket \text{sk}_1 \cdot \beta_0 \rrbracket_0 = \llbracket \tilde{\text{sk}}_1 \cdot \beta_0 \rrbracket_0 + \beta_0 \cdot \gamma_{\text{sk}_1}$$

6. Overall, we obtain that:

$$\llbracket r_1 \cdot \beta_0 \rrbracket_0 + \llbracket r_1 \cdot \beta_0 \rrbracket_1 = r_1 \cdot \beta_0 , \quad \llbracket \text{sk}_1 \cdot \beta_0 \rrbracket_0 + \llbracket \text{sk}_1 \cdot \beta_0 \rrbracket_1 = \text{sk}_1 \cdot \beta_0 .$$

□

Hyb₄ and Hyb₅ are identical. The difference is in the definition of β_1 and u_1 . In **Hyb₄**, β_1 is chosen uniformly at random by $\mathcal{F}_{\text{RVOLE}}$, and u_1 is defined by the honest P_1 :

$$u_1 = r_1 \cdot \beta_1 + \llbracket r_1 \cdot \beta_0 \rrbracket_1 + \llbracket r_0 \cdot \beta_1 \rrbracket_1$$

Moreover, the simulator defines:

$$u_0 = r_0 \cdot \beta_0 + \llbracket r_0 \cdot \beta_1 \rrbracket_0 + \llbracket r_1 \cdot \beta_0 \rrbracket_0 .$$

According to Claim C.3, we conclude that:

$$\begin{aligned} u_0 + u_1 &= r_0 \cdot \beta_0 + r_0 \cdot \beta_1 + r_1 \cdot \beta_0 + r_1 \cdot \beta_1 \\ &= (r_0 + r_1) \cdot (\beta_0 + \beta_1) . \end{aligned}$$

It also holds that In **Hyb₅**, u_1 is chosen uniformly at random, the simulator computes u_0 , and then sets

$$\beta_1 = (u_0 + u_1) \cdot (r_0 + r_1)^{-1} - \beta_0 .$$

There is a one-to-one correspondence between the two values β_1 and u_1 , given all the other values, $u_0, u_1, \beta_0, r_0, r_1$.

Hyb₅ and Hyb₆ are identical. The difference is in the value of w_1 . In **Hyb₆**, we set:

$$w_1 = s \cdot (u_0 + u_1) - w_0 .$$

In **Hyb₅** it is computed as

$$w_1 = H(m) \cdot \beta_1 + r^x \cdot v_1$$

From Claim C.3, it holds that $u_0 + u_1 = r \cdot \beta$, and $v_0 + v_1 = \text{sk} \cdot \beta$, and so:

$$\begin{aligned} w_0 + w_1 &= H(m) \cdot \beta_0 + r^x \cdot v_0 \\ &\quad + H(m) \cdot \beta_1 + r^x \cdot v_1 \\ &= H(m) \cdot (\beta_0 + \beta_1) + r^x \cdot (v_0 + v_1) \\ &= H(m) \cdot \beta + r^x \cdot \text{sk} \cdot \beta \\ &= r \cdot \beta \cdot (r^{-1}(H(m) + r^x \cdot \text{sk})) \\ &= r \cdot \beta \cdot s \\ &= (u_0 + u_1) \cdot s . \end{aligned}$$

Therefore, there is a one-to-one correspondence between w_1 and s given that the other values are fixed.

Hyb₆ and Hyb₇ are identical. First note that the first message $(\Gamma_{r_1}, \Gamma_{\text{sk}_1}, \gamma_{r_1}, \gamma_{\text{sk}_1})$ was computed using information that was sent from the adversary. Moreover, the message (u_1, w_1) was computed with u_1 sampled at random, and w_1 was computed using s , and there is no need to know r_1, sk_1 explicitly. The only place where r_1 is needed is when it sends the opening of the commitment R_1 .

In **Hyb₇**, the trusted party picks r uniformly at random, and computes σ using sk . It sends σ to the simulator, which computes R from σ . Instead of using $R_1 = r_1 \cdot G$, it uses $R_1 = R - R_0$. Clearly, there is a one-to-one correspondence between the two (i.e., instead of P_1 choosing r_1 and setting $r = r_0 + r_1$, we let the trusted party choose r and derive R_1).

Moreover, instead of computing s using the extracted sk , the trusted party does so, where s is computed in the exact same way between the two hybrids.

Hyb₇ and Hyb₈ are identical. This is just a syntactic change.

□